



Generative AI Academy

Language / Text Generative Models

Transformers

Transformers are a type of neural network that have a **unique ability to recognize long-range connections within sequences**. They are particularly useful for **tasks like generating text**, as the model needs to comprehend the preceding words in order to produce the next one.

The introduction of transformers in 2018 was a groundbreaking moment for the field of natural language processing.



Seminal paper published in 2017

Transformers – key capabilities

Transformers
are able to
learn long-
range
dependencies
in sequences.

Transformers
are able to be
trained on very
large datasets.

Transformers
are able to be
parallelized

Transformer: Attention Mechanism

The boy is holding a blue ball



Consider two sentences:

- The **cat** drank the milk because **it** was hungry.
- The cat drank the **milk** because **it** was sweet.

In the first sentence, the word 'it' refers to 'cat'
in the second it refers to 'milk'.

When the model processes the word 'it', **self-attention** gives the model more information about its meaning so that it can associate 'it' with the correct word.

The	The	The
cat	cat	cat
drank	drank	drank
the	the	the
milk	milk	milk
because	because	because
it	it	it
was	was	was
hungry	hungry	hungry
<u>Input</u>	<u>Score 1</u>	<u>Score 2</u>



Transformer: Next token prediction

The movie is a visually stunning, action-packed, and emotionally resonant thrill ride that will leave you on the edge of the seat from the beginning to end. Overall, the experience was magical.

match

Vocabulary

positive $p = .03$

negative $p = .00001$

...

magical $p = .83$

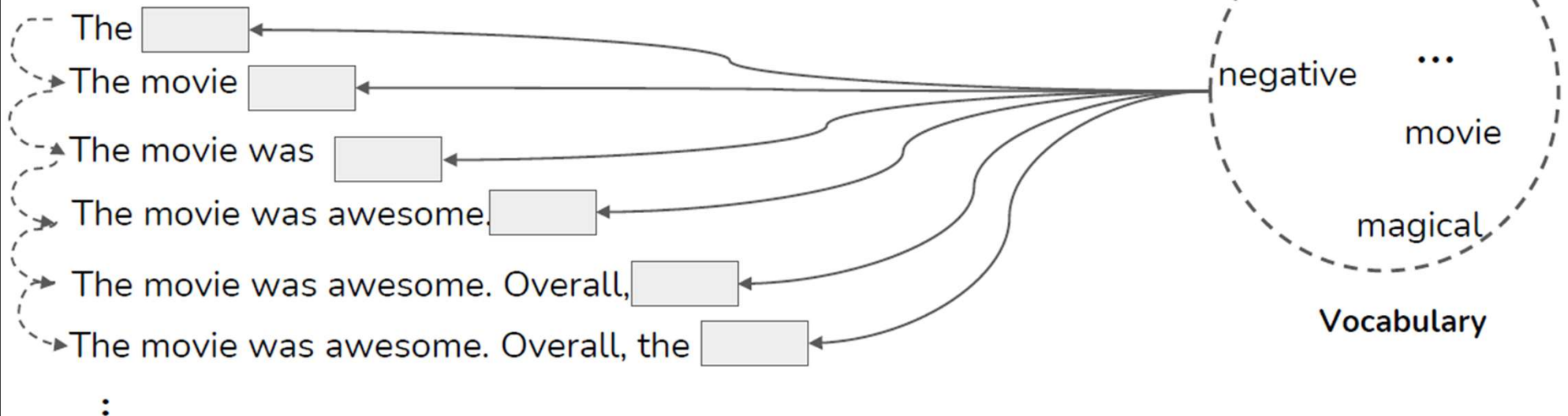
Transformer: **Generate** text with next token prediction (Auto-regressive)

During inference, the LLM predicts the next word in the input sequence.

Input word = prompt

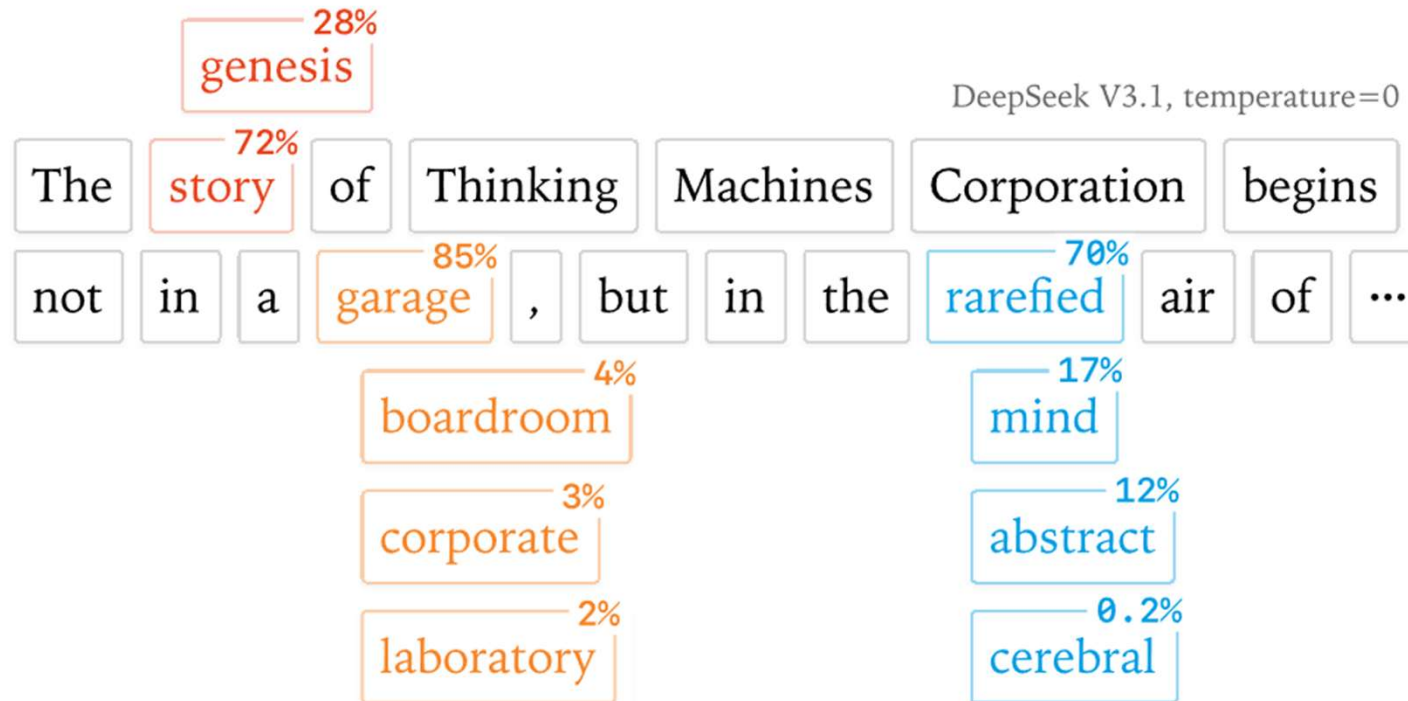
The

Output, word-by-word



Transformer: Next token prediction

DeepSeek V3.1, temperature=0



Transformer: key concepts: Self attention

Stage	"What happens" (plain language)	Why it matters functionally
1 · Three views of every token	For each input vector x_i the layer makes three linear projections: q_i = query (What am I looking for?) k_i = key (What do I contain?) v_i = value (What info can I give?)	Splitting roles lets the model ask: <i>How relevant is token j to token i?</i>
2 · All-to-all similarity check	Compute a dot-product matrix $q_i \cdot k_j$ for every pair (i,j) .	One inexpensive matrix-multiply lets every token "glance" at every other token in parallel (no recurrence).

Transformer: key concepts: Attention Head

A **head** = one *complete attention unit* that includes:

1. Linear projections to create:

- Query (Q)
- Key (K)
- Value (V)

2. The attention computation:

- Score $\rightarrow$ softmax $\rightarrow$ weighted sum

👉 So a head is the **entire pipeline**, not just Q/K/V.

Inside one head

Input embeddings



[Linear layer] $\rightarrow$ Q

[Linear layer] $\rightarrow$ K

[Linear layer] $\rightarrow$ V



Attention(Q, K, V)



Output of ONE head

Multi headed attention

Single-head (one lens)

One set of attention weights tries to capture every relationship (subject-verb, adjective-noun, long-range theme) all at once.

Multi-head (many lenses)

The model splits its “vision” into h smaller lenses (heads). Each head can specialise: one notices grammar, another spots coreference, another tracks long-distance topics, etc.

Transformer: key concepts: Positional encoding

Because self-attention sees the sequence as an unordered bag of vectors, we need a tiny extra hint so

“dog bites man” $\neq$ “man bites dog”.

We simply add or concatenate a position vector:

$$x_i' = \text{token_embedding}_i + \text{position_embedding}_i$$

Feed Forward Neural Network (FFNN) aka Multi Layer Perceptron (MLP)

Multi headed attention – under the hood

1. Copy the same input vectors h times.
2. Give each copy its own tiny pair of glasses (different weight matrices) so it sees the sentence from a unique angle.
3. Let every head do its own attention calculation (build its own weight map).
4. Concatenate the heads—like stacking several transparencies—and run them through one last linear layer so the information blends back together.

Analogy: Think of a detective squad. All detectives read the same case file, but each focuses on a different clue type—fingerprints, alibis, motives. At the end they pool their findings to solve the case.



Summary of next token prediction mechanism

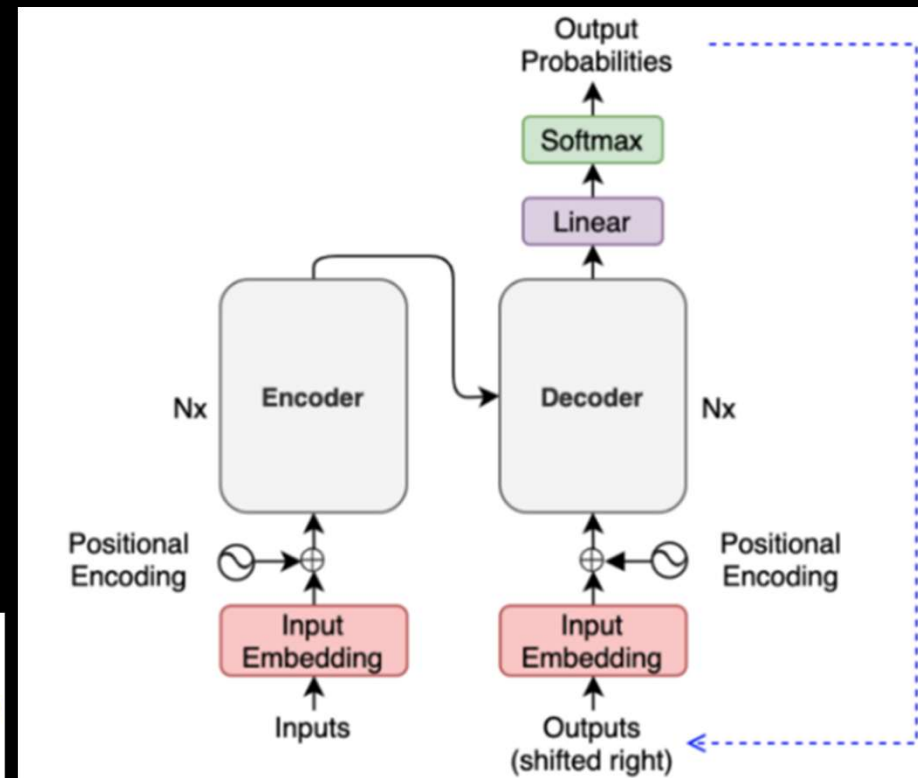
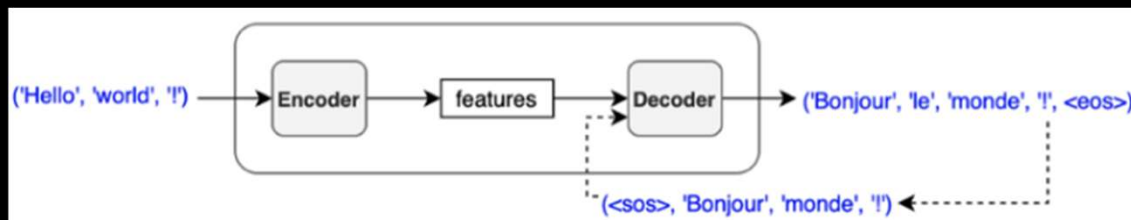
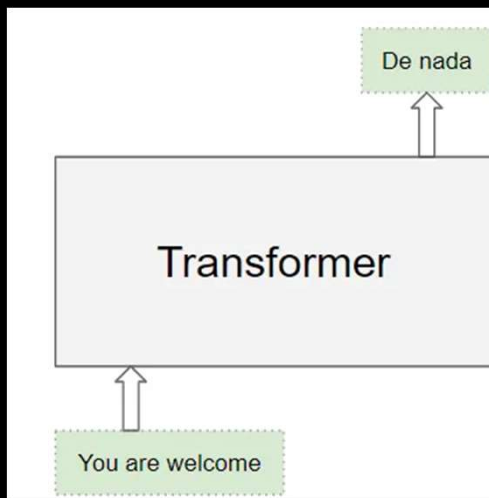
- **Multi-headed attention** computes relevance weights by comparing each token (query) with every other token (keys), producing weighted sums of the value vectors.
- **Feed-forward layers** then transform these per-token representations in a non-linear way, letting the model mix and match features.
- **Determining the “most appropriate” next token** happens in the final step, where the model uses the last position’s hidden state to output logits over the vocabulary. The highest-logit (or sampled) token is picked.

Why both attention *and* FFN are needed

- **Attention** = *context routing*: it decides **what** to bring from earlier tokens and **how much**.
- **FFN** = *feature alchemy*: it turns that routed context into **useful, nonlinear features** that better line up with the vocabulary space for prediction.
- Without attention, the FFN can’t know the sentence is about cricket; without FFN, the model lacks the nonlinear “if-this-then-boost-names” style transformations that make predictions sharp and specific.

Encoders and Decoders (aka seq2seq)

The **encoder** extracts features from an input sentence, and the **decoder** uses the features to produce an output sentence



Encoders and Decoders

Training the model to learn French translation from English.

Training data:

English: The cat sat on the mat

French: Le, chat, s'est, assis, sur, le, tapis

Component	Sequence / Role	Tokenized/ID Representation	Feeds to...	How it's Used
Input (Source X)	The English sentence.	[The, cat, sat, on, the, mat, <EOS>, <PAD>, ...]	Encoder	Encoded into Key (K) and Value (V) vectors.
Encoder Output (K&V)	Contextualized representation of X.	(Dense vector representation of the whole sentence)	Decoder (Cross-Attention)	Provides the context for the decoder to reference.
Target (Decoder Input)	The French translation (shifted right by one).	[<SOS>, Le, chat, s'est, assis, sur, le, tapis, <EOS>]	Decoder (Input)	Used by Masked Self-Attention to form the Query (Q).
Label (Decoder Target)	The true French tokens for comparison.	[Le, chat, s'est, assis, sur, le, tapis, <EOS>, <PAD>, ...]	Loss Calculation	The ground truth the decoder's prediction is compared against.

How encoder works

✓ **Goal: Understand and compress the input.**

→ Input text → tokenized → embedded into vectors.

→ Positional encodings added to preserve order.

→ Multi-Head Self-Attention layers capture relationships between all input tokens.

→ Feed-forward networks refine the meaning.

→ Output: a contextual representation of the entire input sequence.

How decoder works

✓ **Goal: Generate text using both past output and encoder knowledge.**

→ Takes previous output tokens (or start token).

→ Uses Masked Self-Attention to look only at earlier tokens → prevents future leakage.

→ Uses Cross-Attention to attend to encoder outputs.

→ Feed-forward layers refine predictions.

→ Linear + Softmax → produces next token.

Encoders and Decoders

Paradigm	Masking & Direction	Typical Objectives	Example Models
Encoder-only (bidirectional)	Full context each side; no causal mask.	Masked-Language-Model (MLM), contrastive.	BERT , RoBERTa, DeBERTa
Decoder-only (causal)	Left→right; causal mask.	Next-token prediction, RLHF.	GPT-3 , LLaMA-2, Mixtral
Encoder-Decoder (Seq-to-Seq)	Encoder = bidirectional; Decoder = causal + cross-attn.	Denoising, sequence transduction.	T5 , BART, FLAN-T5

Encoders and Decoders

Multi-Head Attention

Multiple learned projections let the model attend to different semantic sub-spaces in parallel.

Positional Encoding:

injects order into self-attention (which is permutation-invariant).

Causal Model:

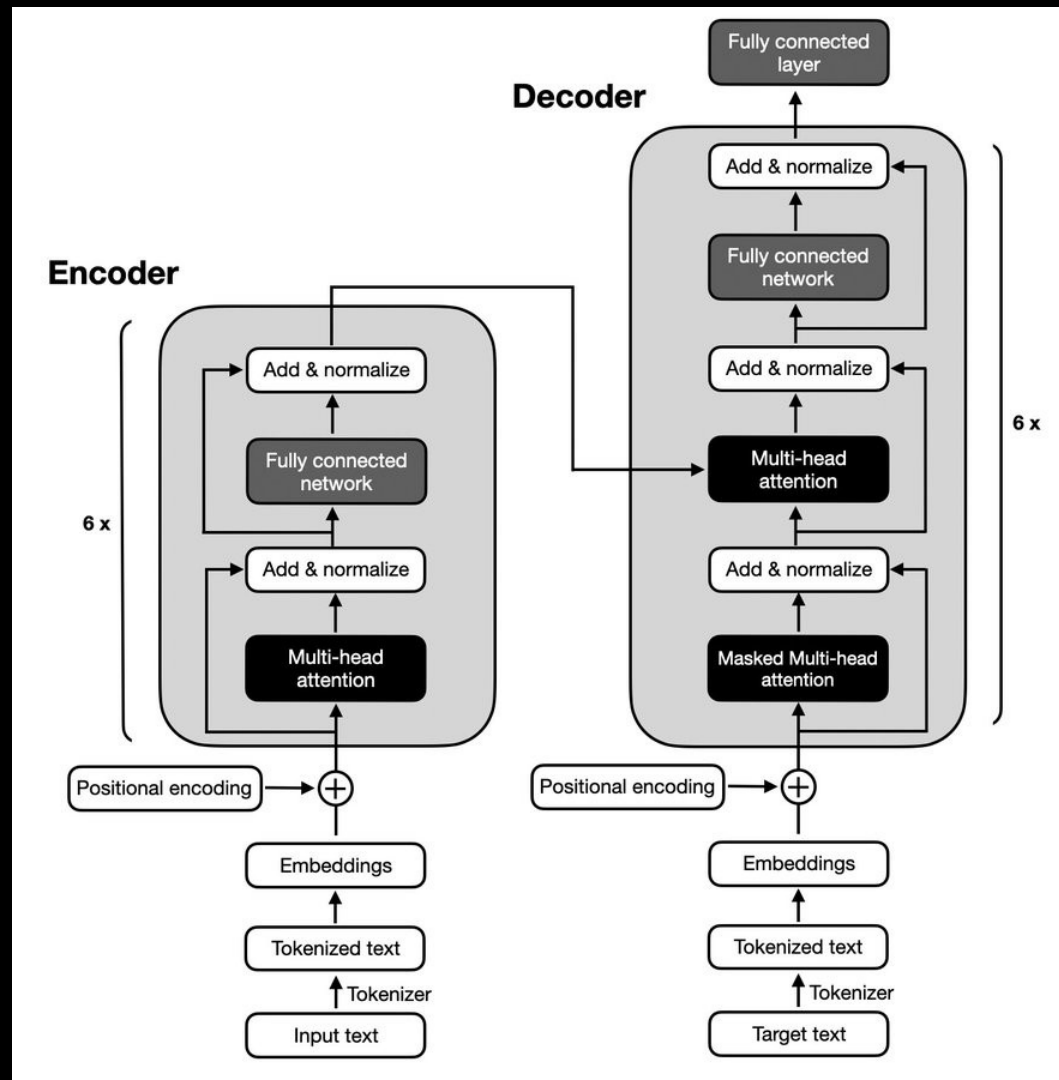
decoder-only transformer where each position can only attend to earlier tokens, enabling generation.

Bidirectional/Masked Model:

encoder-only transformer that can look both left and right to fill in masked tokens; excels at understanding.

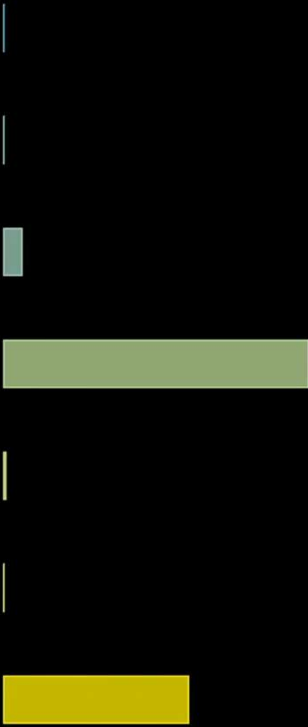
Encoder-Decoder: combines both to translate or transform sequences.

Encoders and Decoders



Convert numbers in the final embedding vector into probabilities

softmax

$$\begin{bmatrix} -0.8 \\ -5.1 \\ +5.7 \\ +8.5 \\ +3.4 \\ -2.2 \\ +8.0 \end{bmatrix} \longrightarrow \begin{bmatrix} e^{x_1} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_2} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_3} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_4} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_5} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_6} / \sum_{n=0}^{N-1} e^{x_n} \\ e^{x_7} / \sum_{n=0}^{N-1} e^{x_n} \end{bmatrix} = \begin{bmatrix} 0.00 \\ 0.00 \\ 0.04 \\ 0.60 \\ 0.00 \\ 0.00 \\ 0.36 \end{bmatrix}$$


Class	Probability
0	0.00
1	0.00
2	0.04
3	0.60
4	0.00
5	0.00
6	0.36

Convert numbers in the final embedding vector into probabilities

Temp = 0

Once upon a time, there was a little girl
named Gold

Gold	100%
Alice	0%
Lily	0%
Emily	0%
Rose	0%
Sarah	0%
Little	0%
Mary	0%
L	0%
Cinder	0%
Lucy	0%
Is	0%
⋮	

Temp = 5

Once upon a time, there was a young and
aspiring web

writer	7%
artist	6%
entrepreneur	5%
musician	5%
author	5%
princess	4%
chef	4%
knight	4%
software	4%
wizard	4%
magician	4%
web	4%
⋮	

Typical training flow

Step 42 318 of training:

- The input pipeline streams 4 096 sequences of 2 048 tokens each (≈ 8 M tokens) onto 128 GPUs.
- Embedding look-ups turn token IDs into vectors; positional encodings are added.
- All 8 M vectors race through the 32 transformer layers in parallel.
- The model outputs a probability for the next token at every position; the loss function compares that to the ground truth already sitting in memory.
- Back-prop computes gradients; GPUs exchange them, average, and adjust weights.
- The optimizer writes the new weights; the step is done.
- New data streams in for step 42 319. None of the old tokens are revisited unless you eventually start a second epoch (rare for 10 T tokens).

Encoder only and Decoder only Models

Aspect	Encoder-Only (BERT-style)	Decoder-Only (GPT-style)
Attention mask	Bidirectional – every token can attend left and right.	Causal (left-to-right) – each token sees only earlier tokens; future positions are masked.
Typical pre-training task	Masked-Language Modeling (MLM): hide 15 % of tokens and predict them.	Next-Token Prediction (autoregressive): predict token $t + 1$ from tokens $\leq t$.
Output style at inference	Produces a <i>fixed-length</i> vector per input token (or pooled sentence vector). Not designed to stream new tokens.	Generates text <i>token-by-token</i> ; can keep extending a sequence indefinitely.
Common downstream uses	• Text classification • Named-entity recognition • Embedding extractor • Reranking / retrieval	• Chatbots, story generation • Code completion • Few-/zero-shot reasoning • Text-to-SQL, translation via prompting

Training flow in Decoder only Model

Prompt tokens (length = L) Generated tokens (so far = T)

Embedding + positional encoding $\rightarrow H_0$ ($(L+T \times d)$)

Masked **self**-attention layer 1 $\rightarrow H_1$

Masked **self**-attention layer 2 $\rightarrow H_2$

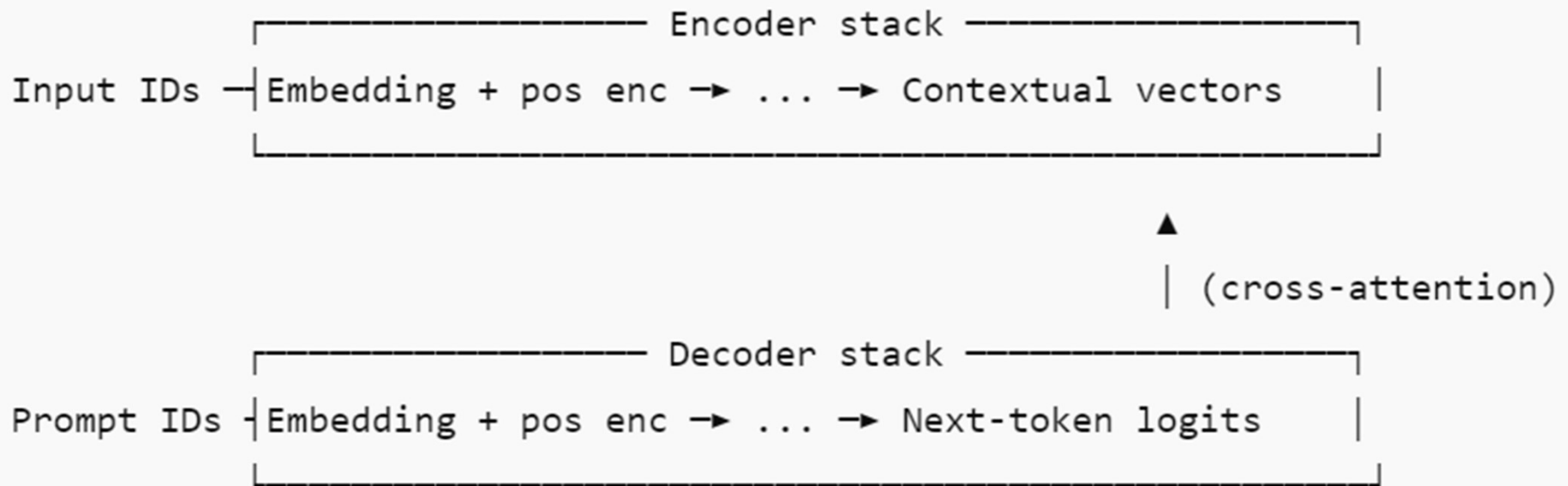
$\vdots$

Masked **self**-attention layer N $\rightarrow H_N$ $\leftarrow$ contextual vectors

Linear head $\rightarrow$ **next**-token logits

MLP = Multi layer perceptron

Training flow in Encoder + Decoder Model



Prompt IDs = token IDs of the text that the decoder has already generated

Encoder + Decoder

Input IDs → Embedding + Positional Encoding → Encoder → Contextual vectors

- **Input IDs** = token IDs of the source text

Example (English):

"I love AI" → [101, 2003, 2293, 9932]

- These go through:
 - **Embedding layer** → converts IDs to vectors
 - **Positional encoding** → adds order information
 - **Encoder stack** → produces **contextual representations**

👉 Output = **contextual vectors** (rich understanding of input)

ENCODER

Prompt IDs → Embedding + Positional Encoding → Decoder → Next-token logits

DECODER

Encoder + Decoder

Task: Translate

Input (encoder): "I love AI"

Target (decoder): "मुझे एआई पसंद है"

During training:

At each step, decoder gets **partial target sentence**:

Step	Decoder Input (Prompt IDs)	Predict
1	<start>	"मुझे"
2	<start> मुझे	"एआई"
3	<start> मुझे एआई	"पसंद"
4	<start> मुझे एआई पसंद	"है"

Cross Attention

- Decoder uses:
 - **Self-attention** → looks at prompt IDs (what it has generated so far)
 - **Cross-attention** → looks at encoder output (input sentence)

👉 This is how it aligns:

- "I" ↔ "मुझे"
- "AI" ↔ "एआई"

Note that the training data contains English-hindi sentence pairs

What would happen without the contextual vector?

What would break without that contextual step?

- **Wrong word choice:** Static embedding might map “Bonjour” near “Bonbon” (candy) or “Bonjourno” (misspelled Italian), confusing the decoder.
- **Poor scaling to longer sentences:** Reordering, agreement, and coreference all rely on rich encoder states; a raw lookup can’t carry that load.
- **Loss of language transfer:** The learned cross-lingual space collapses if encoder vectors aren’t context-aware; the decoder’s queries would have no consistent target to lock onto.

Three concrete benefits of the contextual vector

1. Disambiguation & register

If the French sentence were « *Bonjour, Madame la Présidente* », the encoder's surrounding context would push $H_{\text{enc-Bonjour}}$ toward a *formal* greeting, nudging the decoder to pick "Good morning" instead of the casual "Hello".

2. Polyglot vector alignment

Training forces the decoder's query direction for *greeting* to align with the encoder's *greeting* vectors—independent of language. The shared high-dimensional geometry is why translation works at all.

3. Robustness to typos / sub-word splits

Even if "Bon-jour" appears in sub-word form (`_Bon` , `jour`), self-attention fuses them so the decoder receives one coherent "greeting" signal instead of two half-words.

Encoder + Decoder model often superior for language translation

Encoder-decoder

Bidirectional encoder captures both left & right context → richer source representation.

Cross-attention cleanly separates “read source” from “write target”; easy to train with teacher forcing.

More interpretable alignment (attention heat-maps).

Decoder-only

Source tokens only look *leftwards* within themselves; in practice this is fine but not as expressive as bidirectional encoding.

Source + target share context window → longer sequences, harder length limits, possible interference.

Alignment is implicit in self-attention.

Where decoder only (causal language models) shine?

- **Open-ended text generation**
(stories, marketing copy, creative writing, ideation)
- Chat & multi-turn assistants
- Code completion / synthesis

Popular Language models built on Transformer Architecture

GPT

Generative Pre-trained Transformer

Decoder only

Open AI. GPT-1 released in Jun 2018 (GPT 3.5 in 2022 Nov)

BERT

Bidirectional Encoder Representations from Transformers)

Encoder only, not an LLM (100M odd parameters)

Google. Oct 2018

T5

Encoder and decoder (seq2seq)

Google.

Sentence Transformers

SBERT

- Produces one vector per sentence
- Internally:
 - Operate on tokens, not sentences
 - The sentence embedding is created by combining token embeddings intelligently
 - Common strategies:
 - ◆ A. Mean Pooling (most common) : average of all token embeddings
 - ◆ B. CLS Token (original BERT way) : Use embedding of [CLS] token
 - ◆ C. Max Pooling : Take max value across tokens (dimension-wise)

Generative Text Models

- **Transformers** dominate this space now
- RNN, LSTM, GRU still used depending on use-cases
- VAE and GAN have also been adopted (TextGAN, SeqGAN)
- Advanced architectures such as Memory Augmented Networks, Graph Neural Networks are being researched

How to teach a new language to LLM?

Challenges

The tokenizer cannot segment text meaningfully

Embeddings are random / meaningless

Gradients during fine-tuning are noisy

The model memorizes, not generalizes

Fine-tuning presumes latent knowledge exists. So just Fine Tuning will not be sufficient.

Continued Pre-training is the comprehensive approach in such situation:

Tokenizer training



Embedding learning



Continued pretraining (**self**-supervised)



(**Optional**) instruction fine-tuning

Autoencoder

Autoencoder learns by learning to reconstruct the original data

If vitals are given, it will compress it (reduce dimensions) and learn to reconstruct the vitals

So when new set of vitals are given, it will compress and then reconstruct using learnt weights
The reconstruction error will tell us how close or far the new vital is from learnt pattern

Useful for anomaly detection

Cross Encoder

Cross Encoder is an **Encoder only model** except that the input sequences are fed in a certain manner
Useful for re-ranking

If you have:

- Query: *"GLP-1 coverage"*
- Top-3 retrieved docs: D1, D2, D3

You do **this**, not a single combined input:

Pass 1: [CLS] Query [SEP] D1 [SEP] → Score1

Pass 2: [CLS] Query [SEP] D2 [SEP] → Score2

Pass 3: [CLS] Query [SEP] D3 [SEP] → Score3

👉 Then:

- Sort scores → re-ranked results

Transformer and Encoder variants

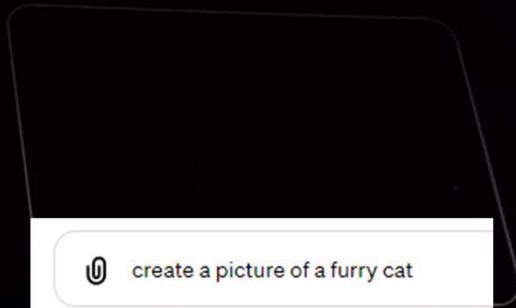
Aspect	Encoder	Bi-Encoder	Cross-Encoder	Autoencoder	Encoder–Decoder
Core Idea	Encode one sequence	Encode two sequences <i>independently</i>	Encode two sequences <i>jointly</i>	Compress → reconstruct same input	Encode input → generate output
Architecture	Single encoder (e.g., BERT)	Two encoders (shared weights)	Same encoder (joint input)	Encoder + decoder (same input/output)	Encoder + decoder (different input/output)
Input Format	Single text	Query → Encoder, Doc → Encoder	[Query + Doc] together	Single input	Input → Output sequence
Attention Type	Self-attention	Self-attention (independent)	Self-attention (cross interaction)	Self-attention	Self + cross-attention
Cross Interaction	✗ No	✗ No (only via similarity)	✓ Yes (token-level interaction)	✗ No	✓ Yes (encoder ↔ decoder)
Output	Embedding / classification	Two embeddings → similarity score	Direct relevance score	Reconstructed input	Generated sequence
Training Objective	Understanding (MLM, classification)	Embedding similarity (contrastive loss)	Relevance / ranking loss	Reconstruction loss	Sequence generation

Aspect	Encoder	Bi-Encoder	Cross-Encoder	Autoencoder	Encoder–Decoder
 Training Paradigm	Self-supervised + supervised fine-tuning	Self-supervised + contrastive supervised fine-tuning	Supervised (labeled relevance pairs)	Self-supervised / unsupervised	Self-supervised + supervised fine-tuning
Speed (Inference)	 Fast	  Very fast (precompute docs)	 Slow (pairwise)	 Moderate	 Slower
Scalability	High	 Very high (vector DB friendly)	Low	Moderate	Moderate
Typical Use Cases	NER, classification	Retrieval, semantic search	Reranking, deep matching	Denoising, anomaly detection	Translation, summarization
Example Models	<u>BERT</u>	SBERT, BGE embeddings	BGE reranker, monoBERT	Denoising autoencoders	<u>T5</u> , BART
Used in RAG?	 Indirect	 Retrieval (first stage)	 Reranking (second stage)	 Rare	 Answer generation
Strength	Strong understanding	Scalable similarity search	High precision matching	Compact representations	Flexible generation
Limitation	No comparison	No deep interaction	Slow, expensive	Not for retrieval/ranking	Overkill for scoring

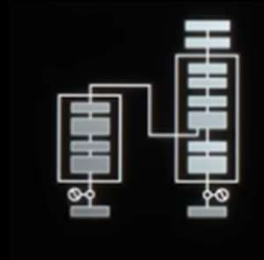
Video: transformers basics

Video: How transformer does prediction / generation of text

Under the hood – generating picture of cat from text



GenAI chatbot - prompt



GPT

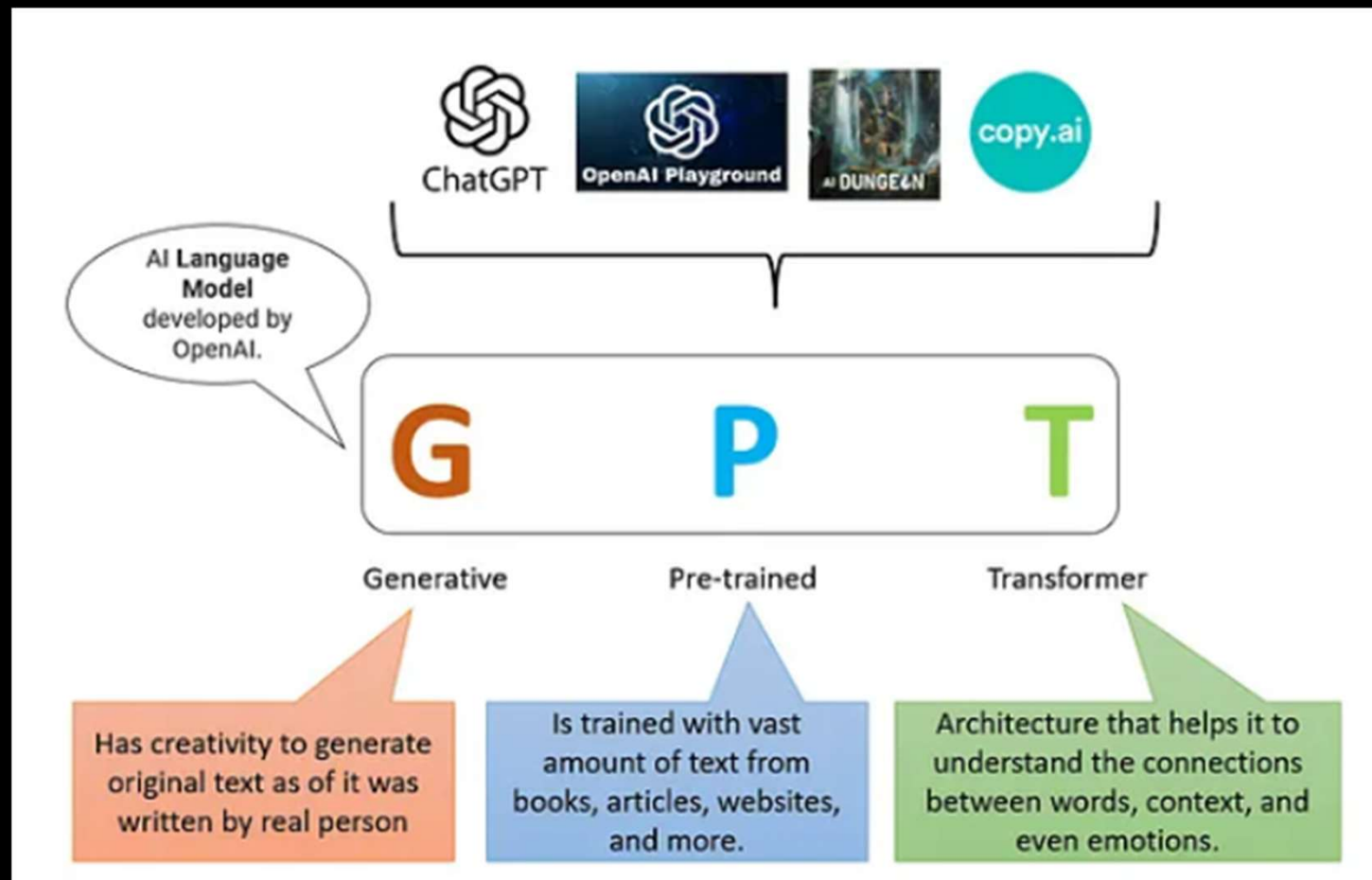


Picture of Cat

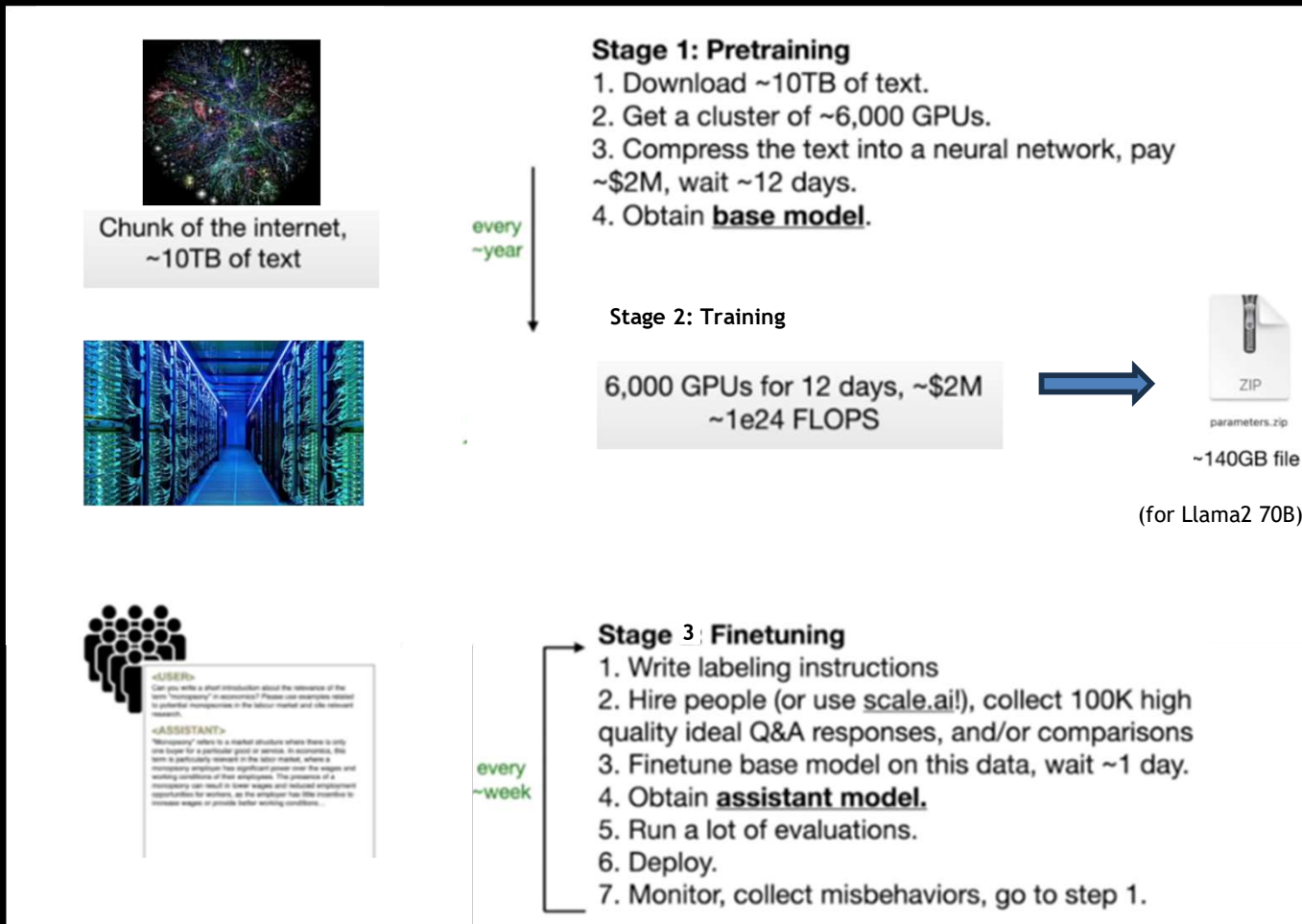
- Features of cat learnt from training data – images of cats (using convolutional neural network) – converted into embeddings
- Text “create picture of...” converted to embeddings using self-attention mechanism (captures dependencies among the input tokens)
- The embeddings are matched to find the common embedding spaces (the text and image will have same embedding positions hence making it possible to generate a picture from text)

Video: Image generation from text - goldfish

GPT



GPT – Llama Model training process

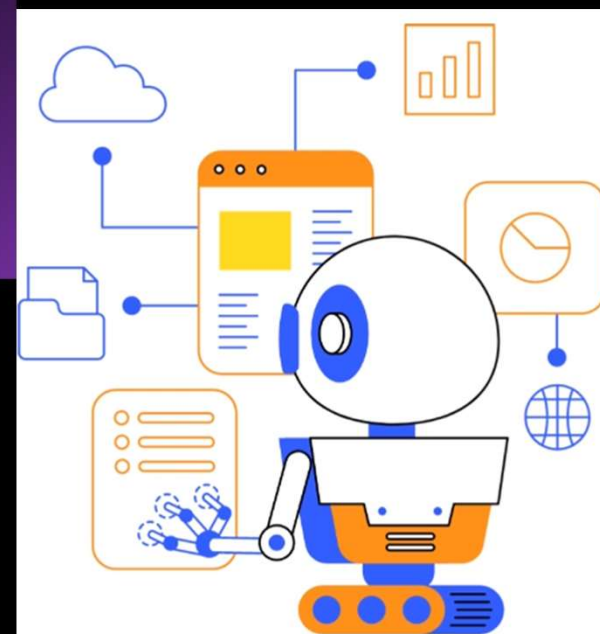


Generative AI

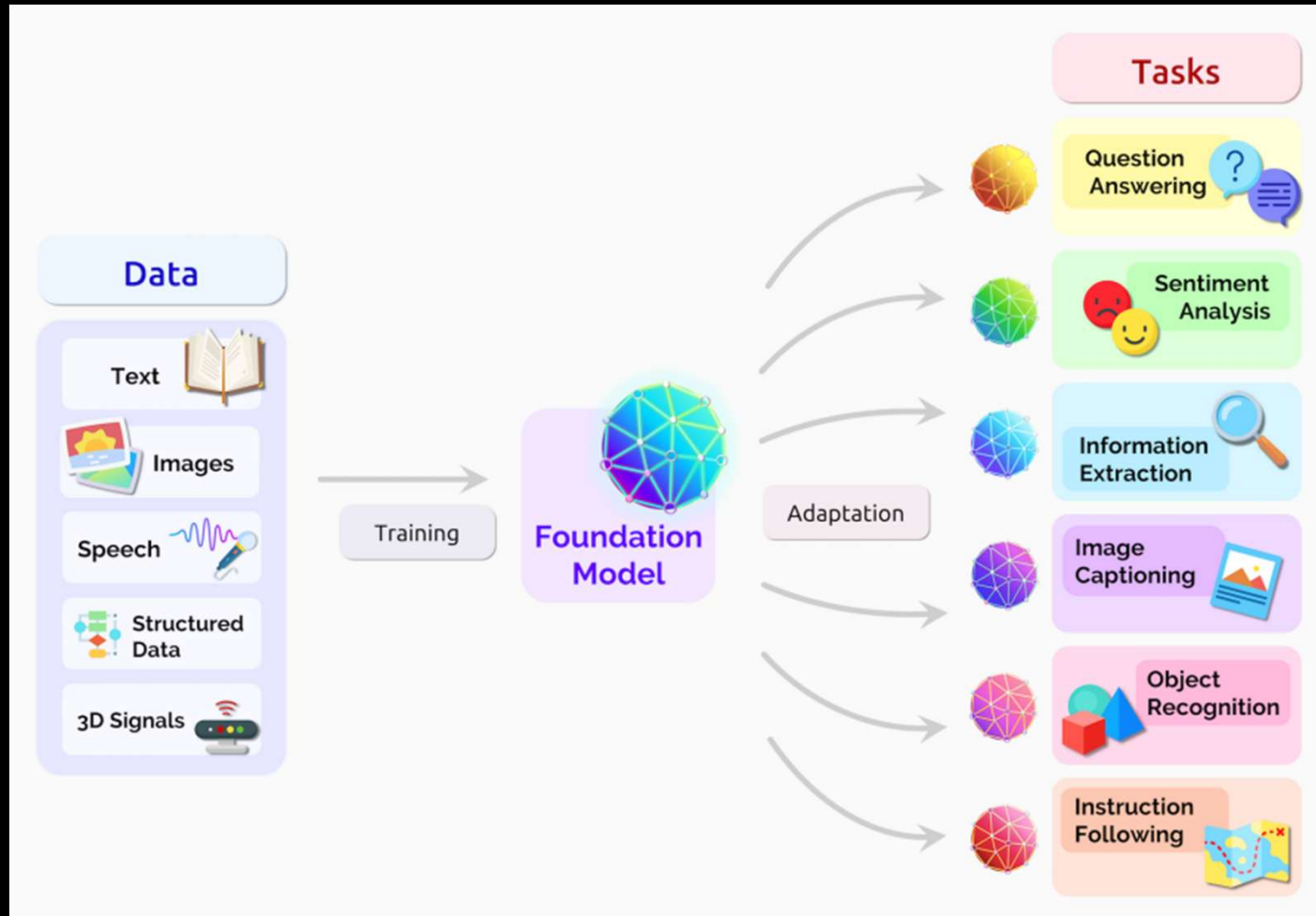
Generative AI

is a type of artificial intelligence that focuses on creating new content. It's a subset of machine learning, drawing from techniques like deep learning and reinforcement learning to generate output that can include text, images, music, video, and more.

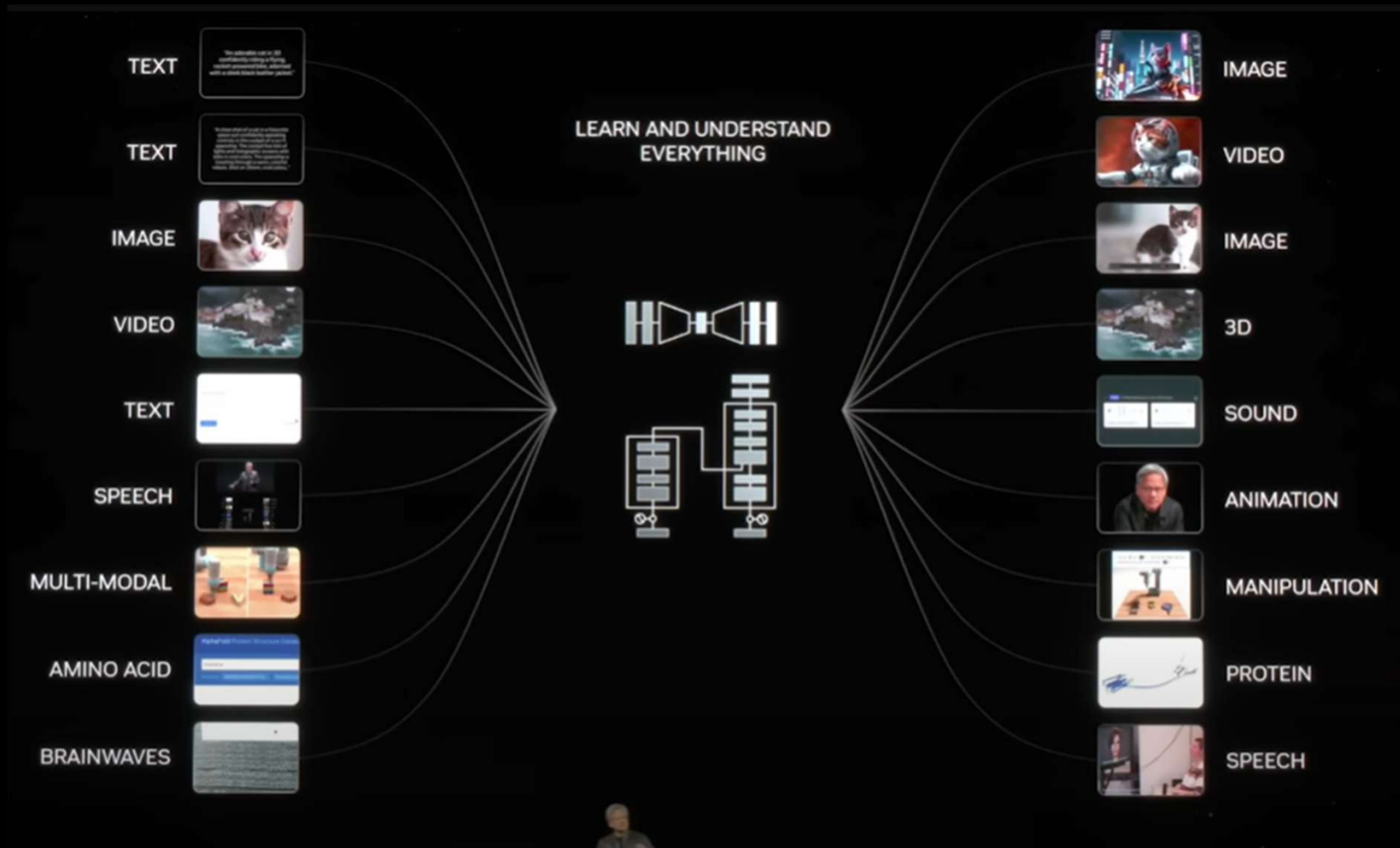
Combination of GPT (or other **generative** models) with a chat interface



Foundation Models



Multi modal models

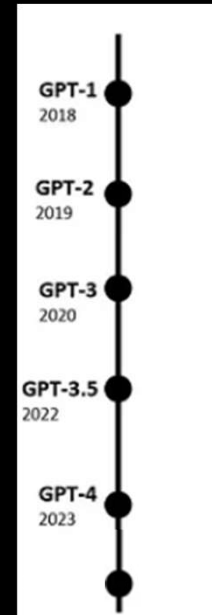


LLM – Large Language Model

General-purpose language model

- **pre-trained** on **massive** data sets
- uses deep learning techniques
- can be **fine-tuned** for specific purposes
- generate natural language outputs

OpenAI



Steps involved in creating a LLM



Chunk of the internet,
~10TB of text



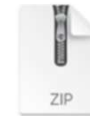
every
~year

Stage 1: Pretraining

1. Download ~10TB of text.
2. Get a cluster of ~6,000 GPUs.
3. Compress the text into a neural network, pay ~\$2M, wait ~12 days.
4. Obtain **base model**.

Stage 2: Training

6,000 GPUs for 12 days, ~\$2M
~1e24 FLOPS



parameters.zip
~140GB file

(for Llama2 70B)



<USER>
Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to particular monopolies in the labour market and the relevant research.

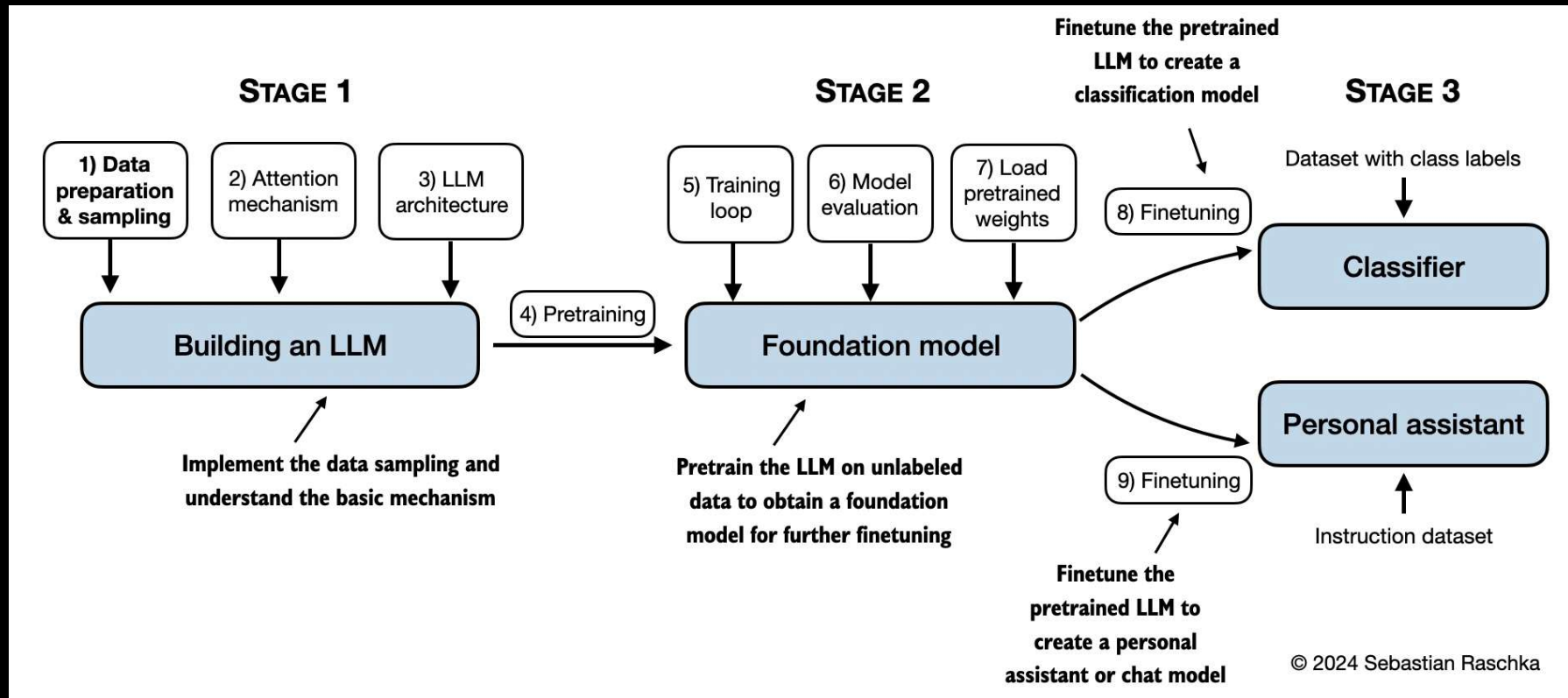
<ASSISTANT>
"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. The presence of a monopsony can result in lower wages and reduced employment opportunities for workers, as the employer has little incentive to increase wages or provide better working conditions.

every
~week

Stage 3: Finetuning

1. Write labeling instructions
2. Hire people (or use [scale.ai](#)!), collect 100K high quality ideal Q&A responses, and/or comparisons
3. Finetune base model on this data, wait ~1 day.
4. Obtain **assistant model**.
5. Run a lot of evaluations.
6. Deploy.
7. Monitor, collect misbehaviors, go to step 1.

Steps involved in creating a LLM



<https://github.com/rasbt/LLMs-from-scratch>

GPT4 training size estimates

GPT4 Model Estimates

Training Size

of Book shelves for 13T tokens

650 kms

Long line of Library Shelves



100000 tokens per Book
100 Books per shelf
2 Shelves per meter

Compute Size

Compute time for 2.15 e25 FLOPs

7 million years

On mid-size Laptop (100GFLOPs)



100GFLOPs per second

Model Size

Size of Excel Sheet for 1.8T params

30,000

Football Fields sized Excel Sheet



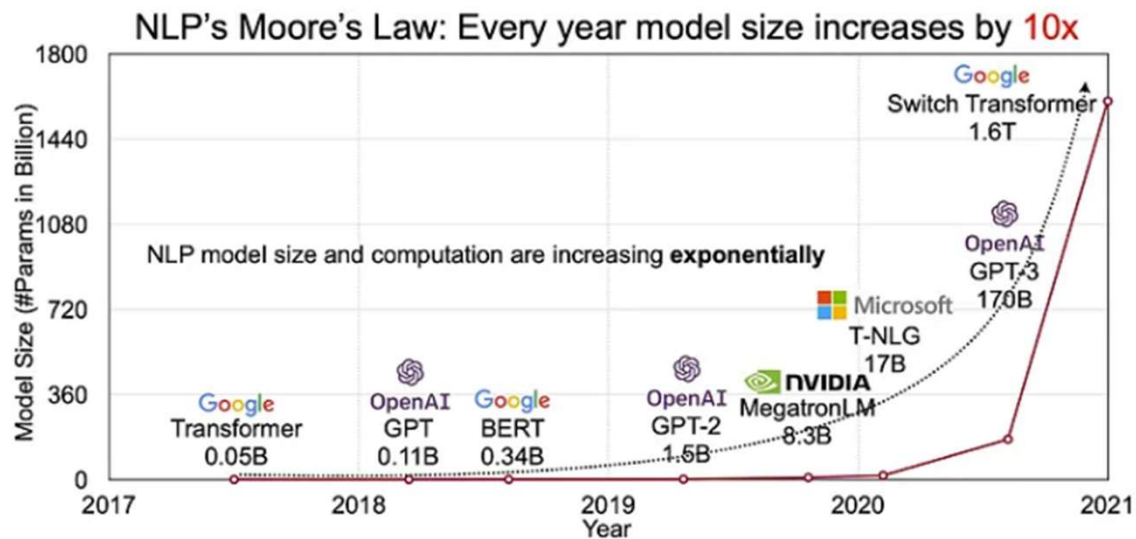
1x1 cm per Excel cell
100 x 60 meters Field Size

Source: <https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked>

GPT 3.5 training resources
and duration
90-100 days to train on
25,000 Nvidia A100 GPUs

3,125 servers
25,000 GPUs
3,125 * 14,040 to 16,900
KWh
**43,875,000 to 52,812,500
KWh**

GPT Model size



Chat GPT 3.5
175 billion parameters

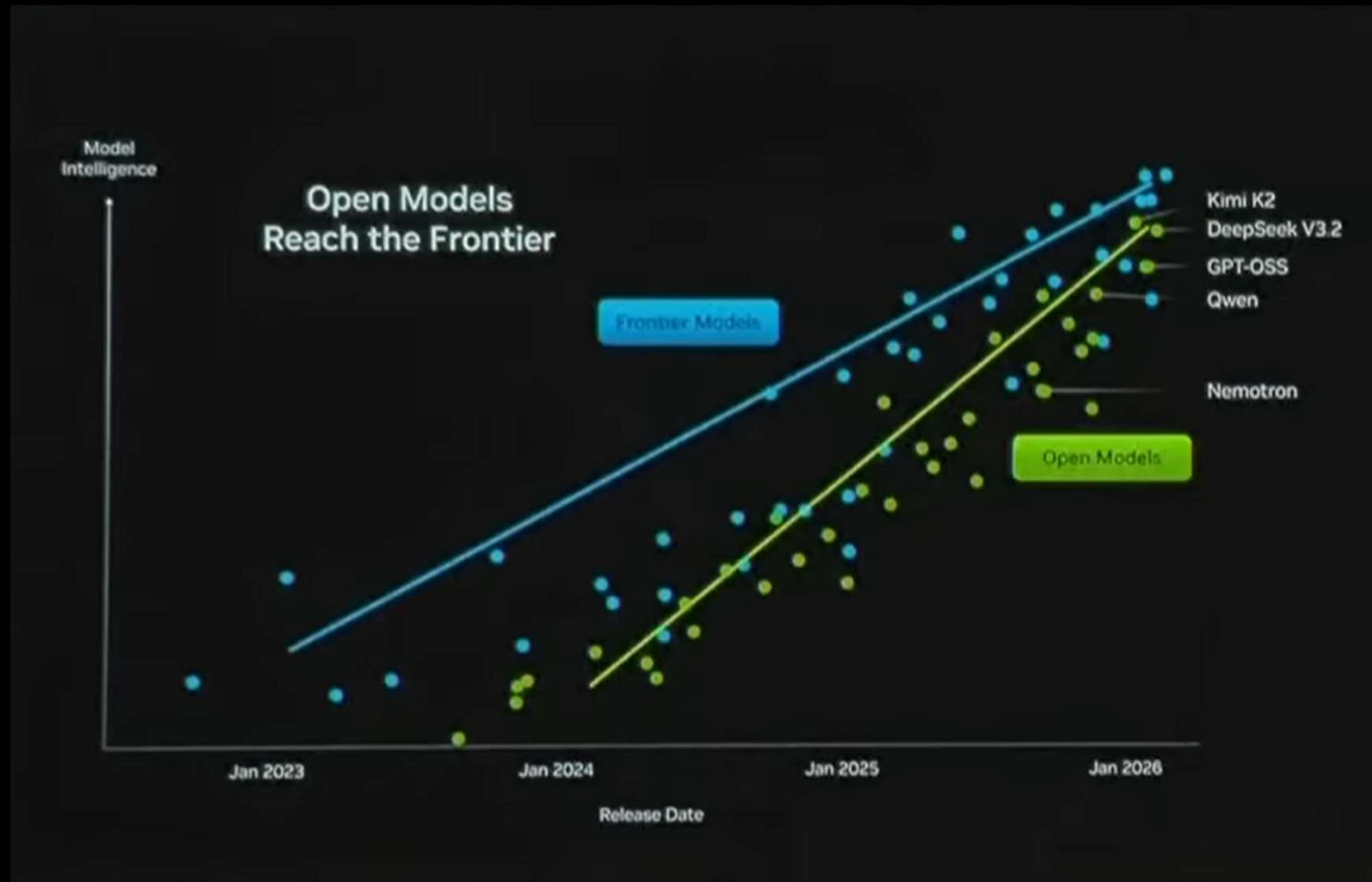


Chat GPT 4
~170 trillion parameters
(exact figure unconfirmed)

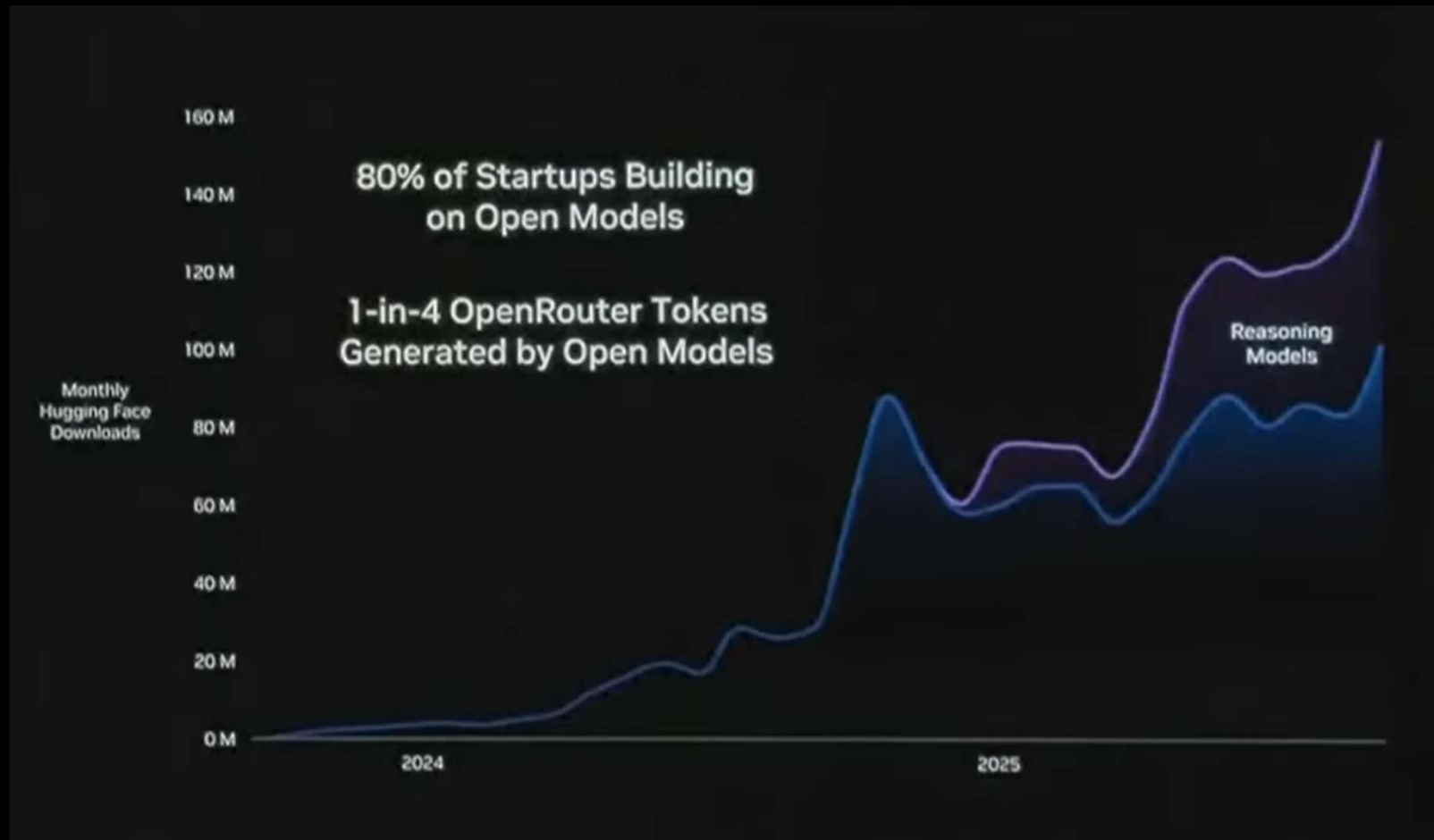
LLM offerings have mushroomed in less than 2 years!



Jensen at CES 2026



Jensen at CES 2026



Generative Image Architectures

- Generative Adversarial Network (GAN)

Invented: 2014 **By:** Ian Goodfellow et al

- Variational Auto Encoder (VAE)

Invented: 2013 **By:** Kingma, Diederik P., and Max Welling

- Stable Diffusion

Invented: 2022 **By:** Stability AI – led by Emad Mostaque

Paper: *"High-Resolution Image Synthesis with Latent Diffusion Models"*

- Vision Transformers (ViT)

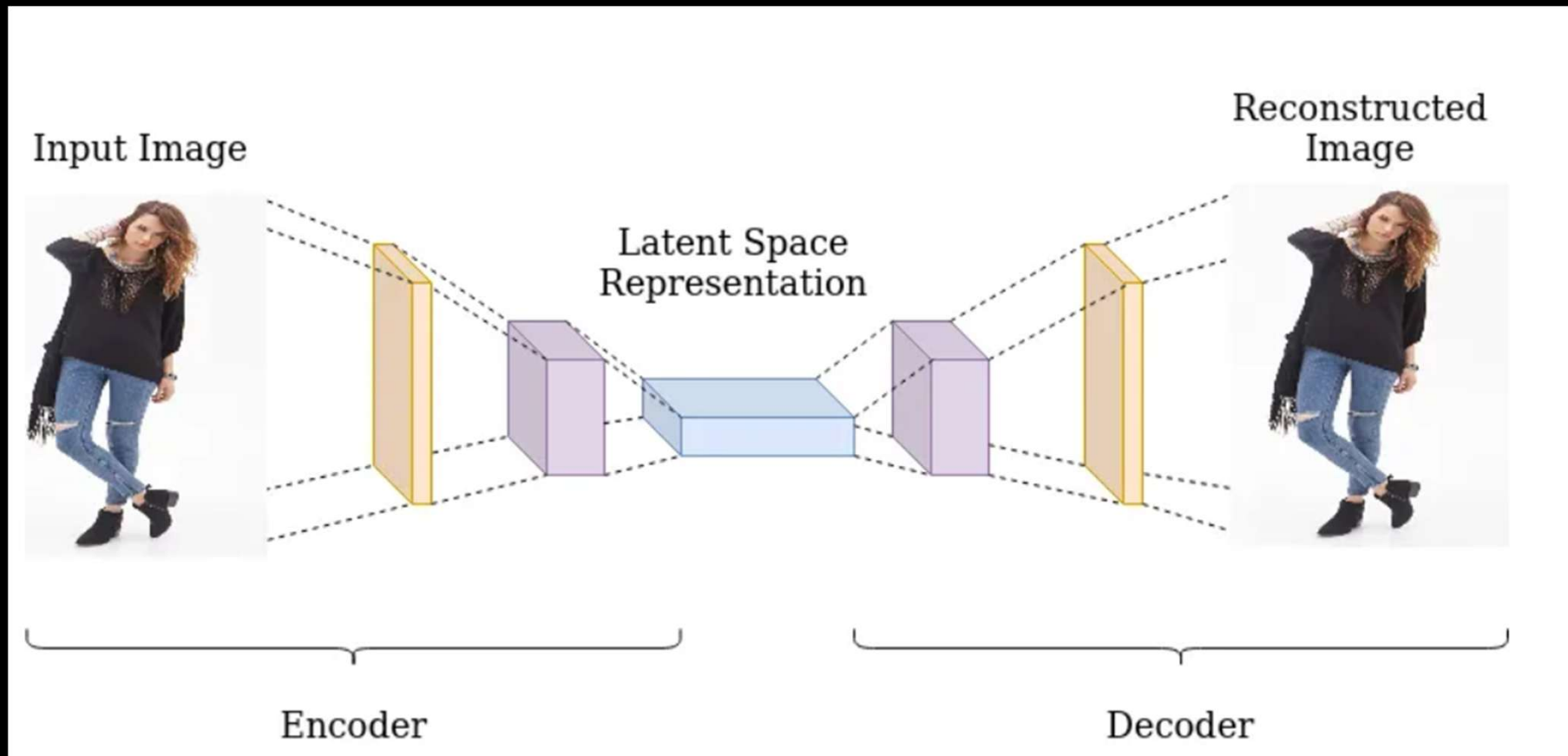
- DALL-E

- Midjourney

Popular image generation models

- DALL-E by Open AI
- Midjourney by Midjourney
initial version: 2022, v6: Dec 2023
- Stable Diffusion XL
Opensource
- Imagen
Google
- Firefly
By Adobe

Encoders and decoders



Encoding techniques: PCA and Autoencoders

Feature	PCA (Principal Component Analysis)	Autoencoders
Supervision	Linear transformation technique Unsupervised (no labels required)	Non-linear neural network-based encoding Unsupervised (self-supervised learning)
Feature Extraction	Identifies linear correlations between features	Learns complex, non-linear feature representations
Dimensionality Reduction	Reduces dimensions by projecting data onto principal components	Compresses data using a neural network bottleneck
Computational Complexity	Low (based on eigenvalue decomposition/SVD)	High (requires training a neural network)
Interpretability	Easy to interpret (principal components are linear combinations of features)	Hard to interpret (learned latent space is abstract)
Non-linearity Handling	✗ Cannot capture non-linear relationships	✓ Captures complex, non-linear patterns
Scalability	Efficient for small to medium datasets	Scales well but requires more computational power
Data Reconstruction	Approximate reconstruction (lossy) using principal components	Can reconstruct input but may not be perfect (depends on loss function)
Flexibility	Fixed transformation (does not adapt to new data)	Can be trained on specific datasets and fine-tuned
Robustness to Noise	Sensitive to noise in data	Denoising autoencoders can effectively remove noise

Encoding and Decoding example

Example: Image Compression

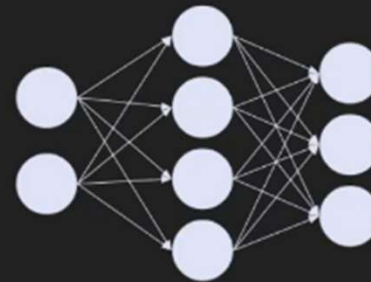
Consider a high-resolution image (128×128 pixels).

Instead of storing all 16,384 pixels (128×128), an autoencoder might encode it into a vector of just 100 numbers.

The decoder then tries to reconstruct the original image from just those 100 numbers.

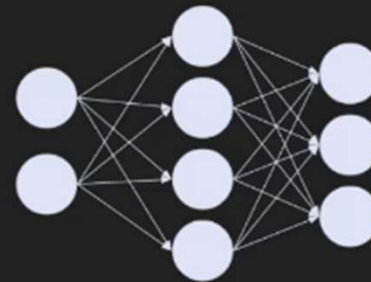
Discriminative and Generative Models

Discriminative models
classify or characterize
existing data.

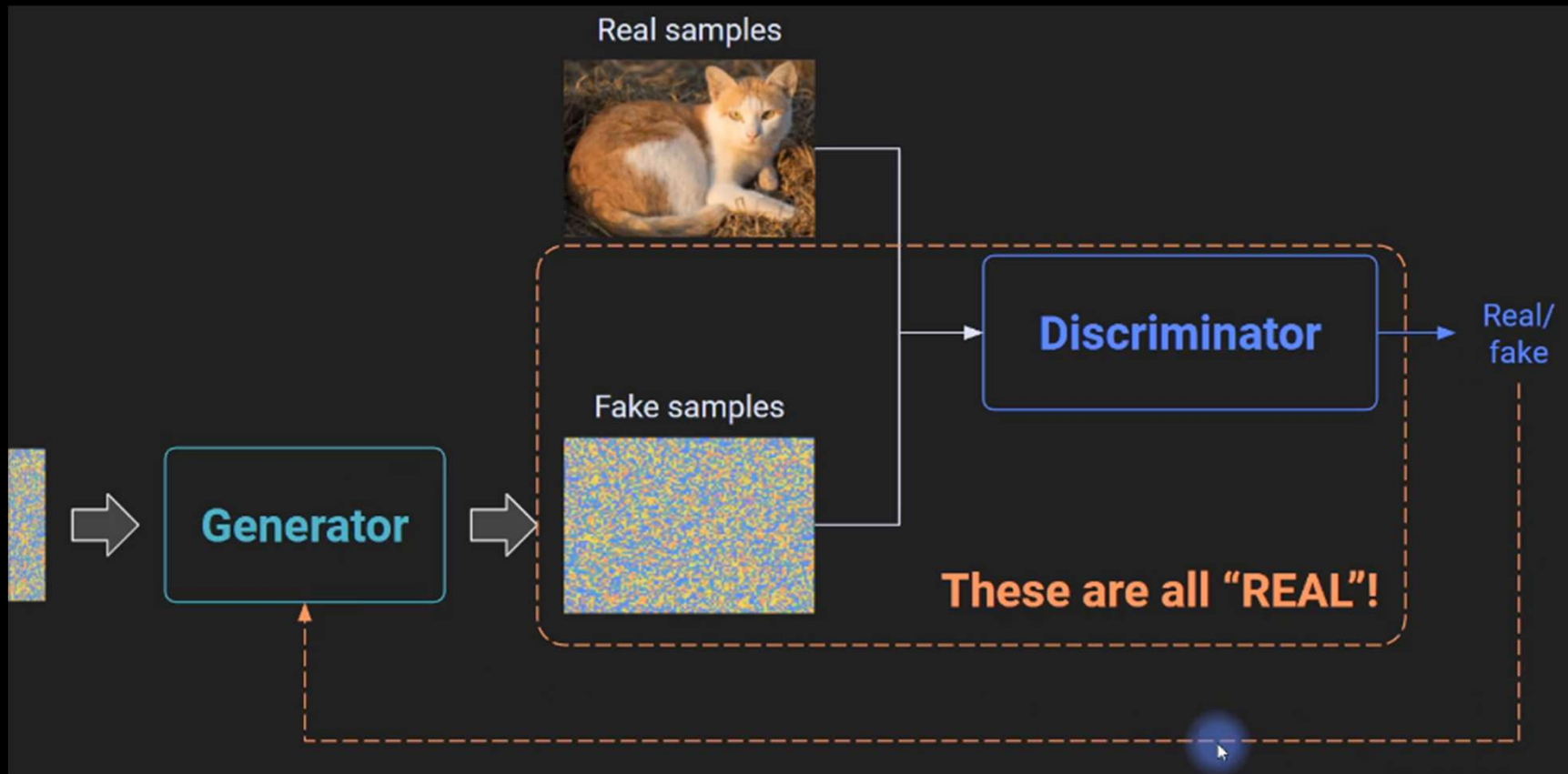


It's a cat.

Generative models
create new samples.



Generative Adversarial Network (GAN)

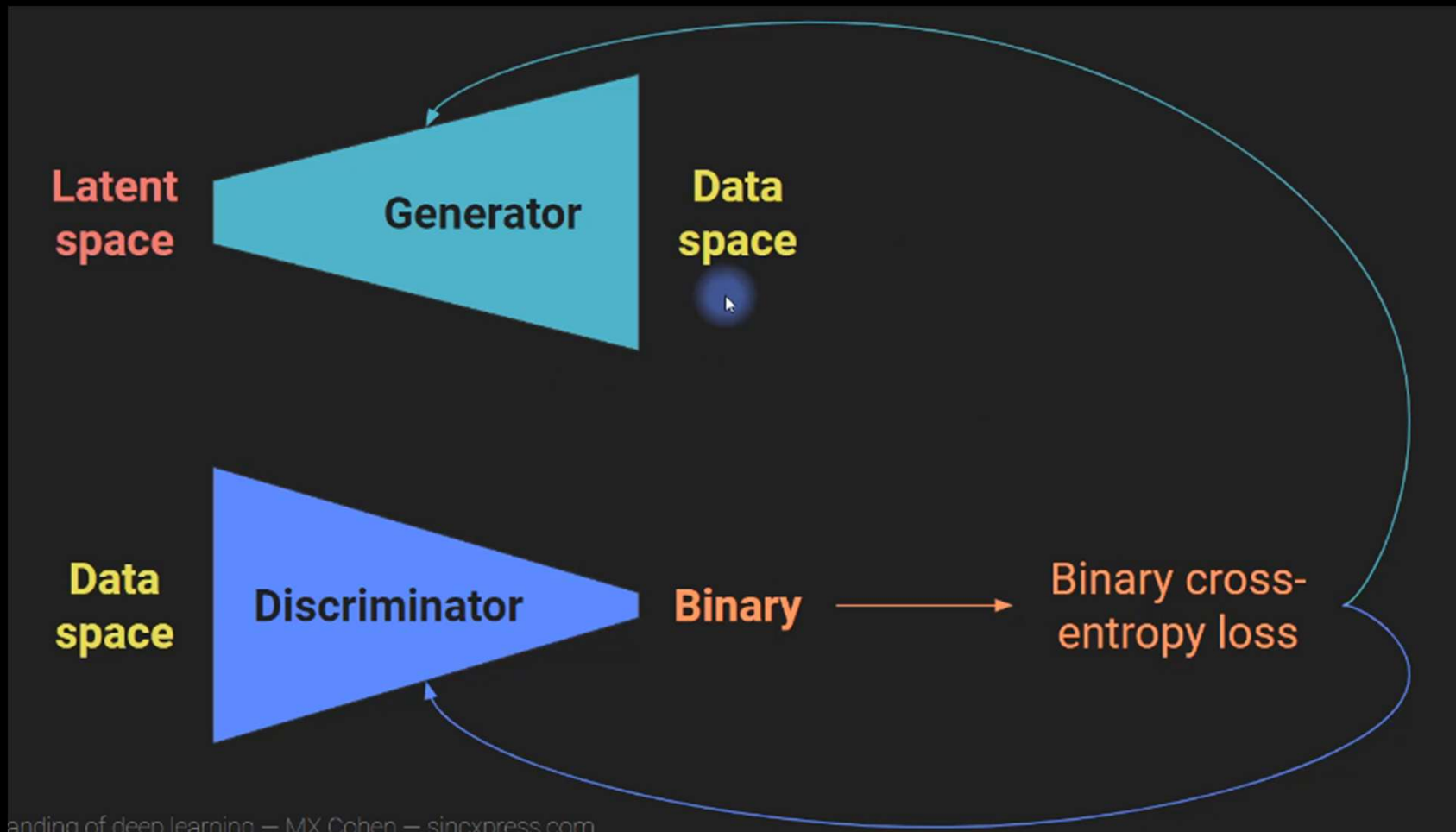


Adversarial Network / training

Discriminator model training (we give the model labelled data – real images)

Generator model starts from noise and corrects itself by backpropagation to come close to real sample

Generative Adversarial Network (GAN)



Applications of GAN

Combine two categories to increase training set.

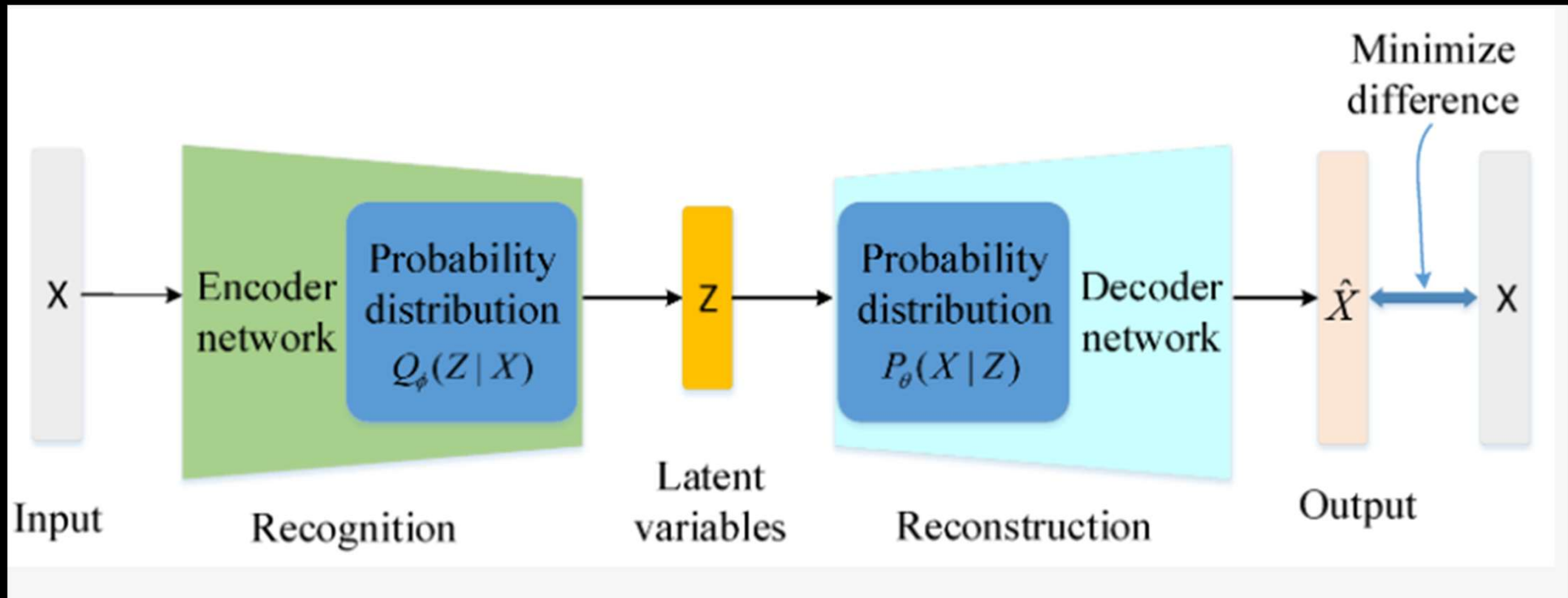
Super-resolution upscaling.

Personalized medical devices.

Evil (e.g., deep fakes).

Learning from private data without privacy concerns
(learn from data, then delete original data).

Variational Auto Encoder (VAE)



GAN Vs VAE

Feature	VAE (Variational Autoencoder)	GAN (Generative Adversarial Network)
Architecture	Encoder-Decoder	Generator-Discriminator (Adversarial)
Learning Approach	Probabilistic (learns latent space distribution)	Game-theoretic (two networks compete)
Latent Space Representation	Continuous, structured, and smooth	Unstructured, often chaotic
Training Objective	Maximizes log-likelihood with KL-divergence regularization	Minimax game (Generator vs. Discriminator)
Optimization	Uses reconstruction loss + KL divergence	Uses adversarial loss (binary cross-entropy)
Sample Quality	Lower quality, but smooth interpolation	High quality, but may have mode collapse
Stability of Training	More stable, but blurry images	Less stable, often difficult to train
Mode Collapse Issue	✗ No mode collapse	✓ Can suffer from mode collapse (generator produces limited variety)

VAE Demo and assignment

1. Input Image:

(28, 28, 1) (grayscale image)

2. Encoder Output:

- Mean (μ): [1.0, -0.5]
- Log Variance ($\log(\sigma^2)$): [0.1, 0.2]

3. Latent Vector (Sampled): Stochastic encoding

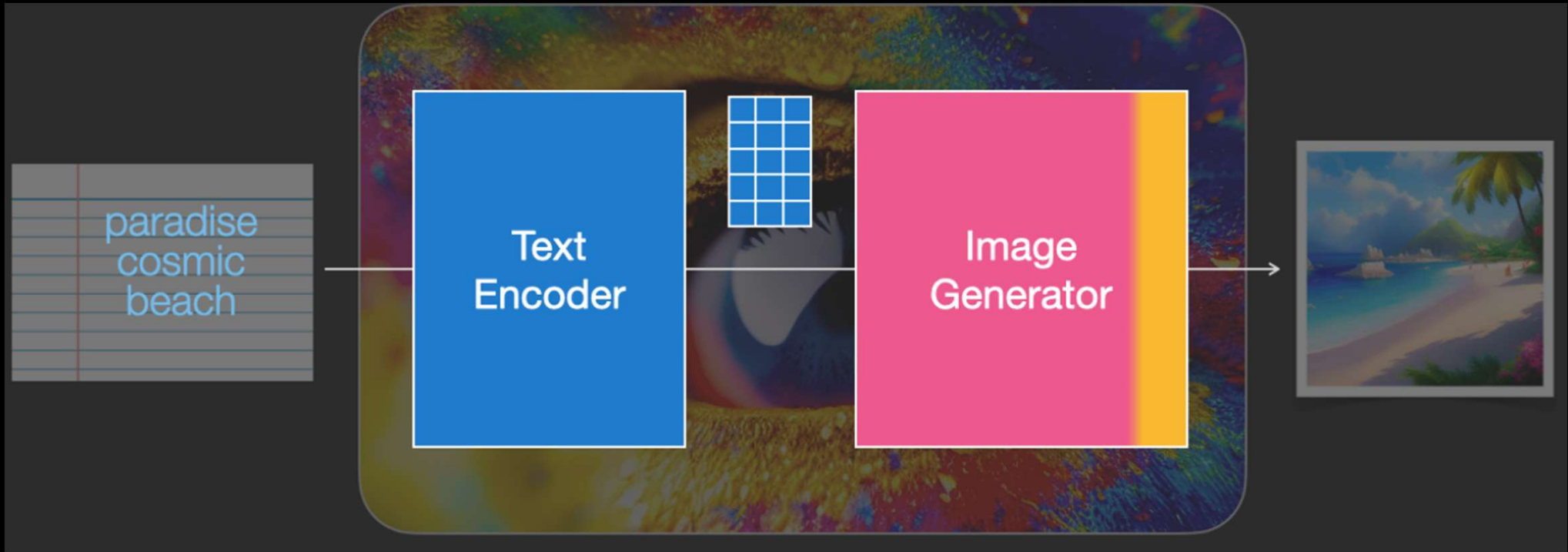
$$z = \mu + \sigma \cdot \epsilon$$

Example: [1.2, -0.6]

4. Decoder Output:

Reconstructed image of shape (28, 28, 1).

Stable Diffusion



<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion training dataset creation

Training examples are created by generating **noise** and adding an **amount** of it to the images in the training dataset (forward diffusion)

1
Pick an image



2
Generate some
random **noise**



Noise sample 1

3
Pick an amount
of **noise**

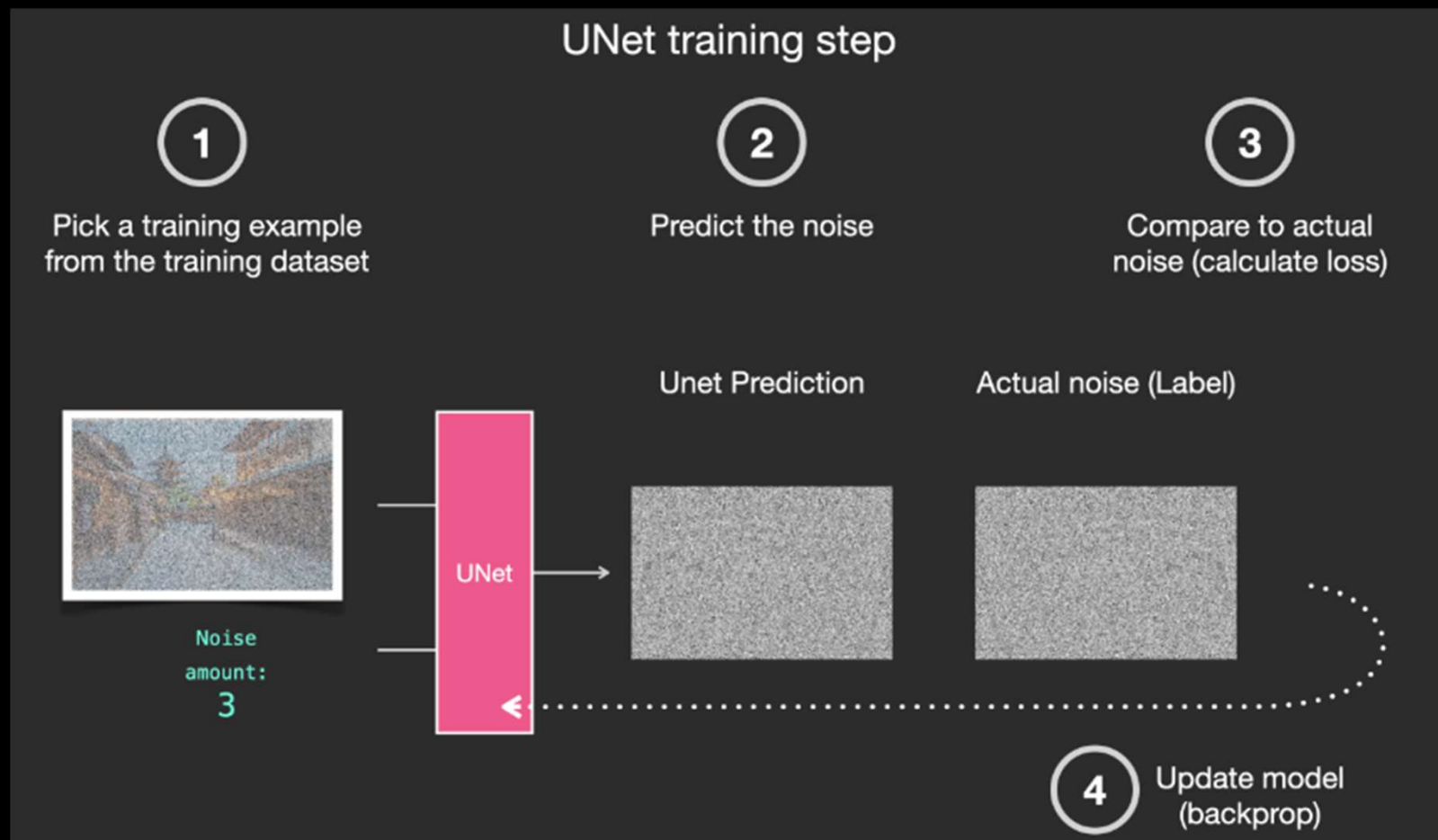


4
Add **noise** to
the image in
that **amount**



<https://jalammar.github.io/illustrated-stable-diffusion/>

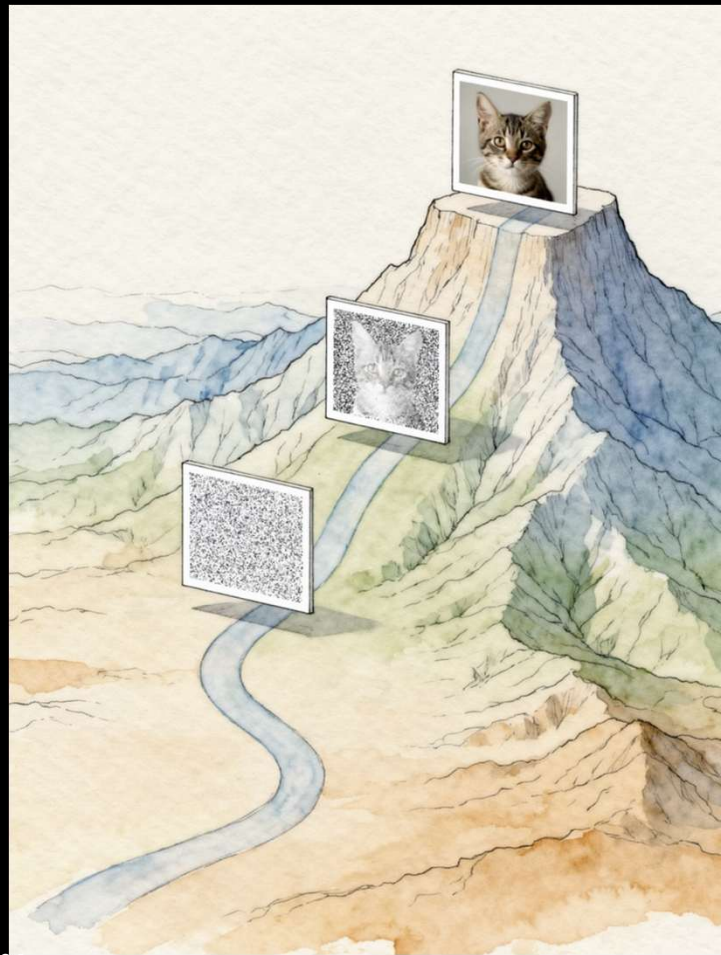
Stable Diffusion training



<https://jalammar.github.io/illustrated-stable-diffusion/>

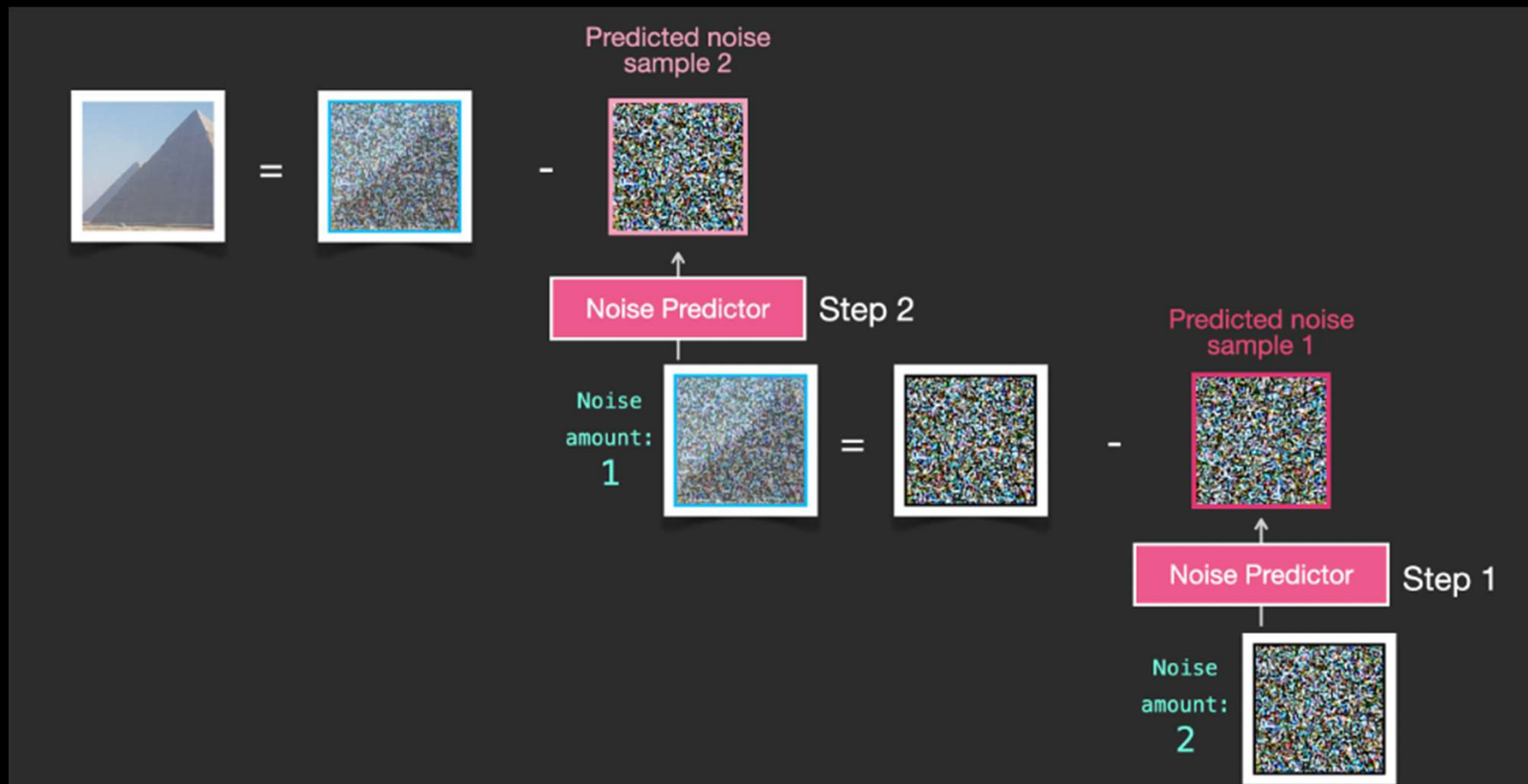
Stable Diffusion – image generation by de-noising

Man's search for meaning!



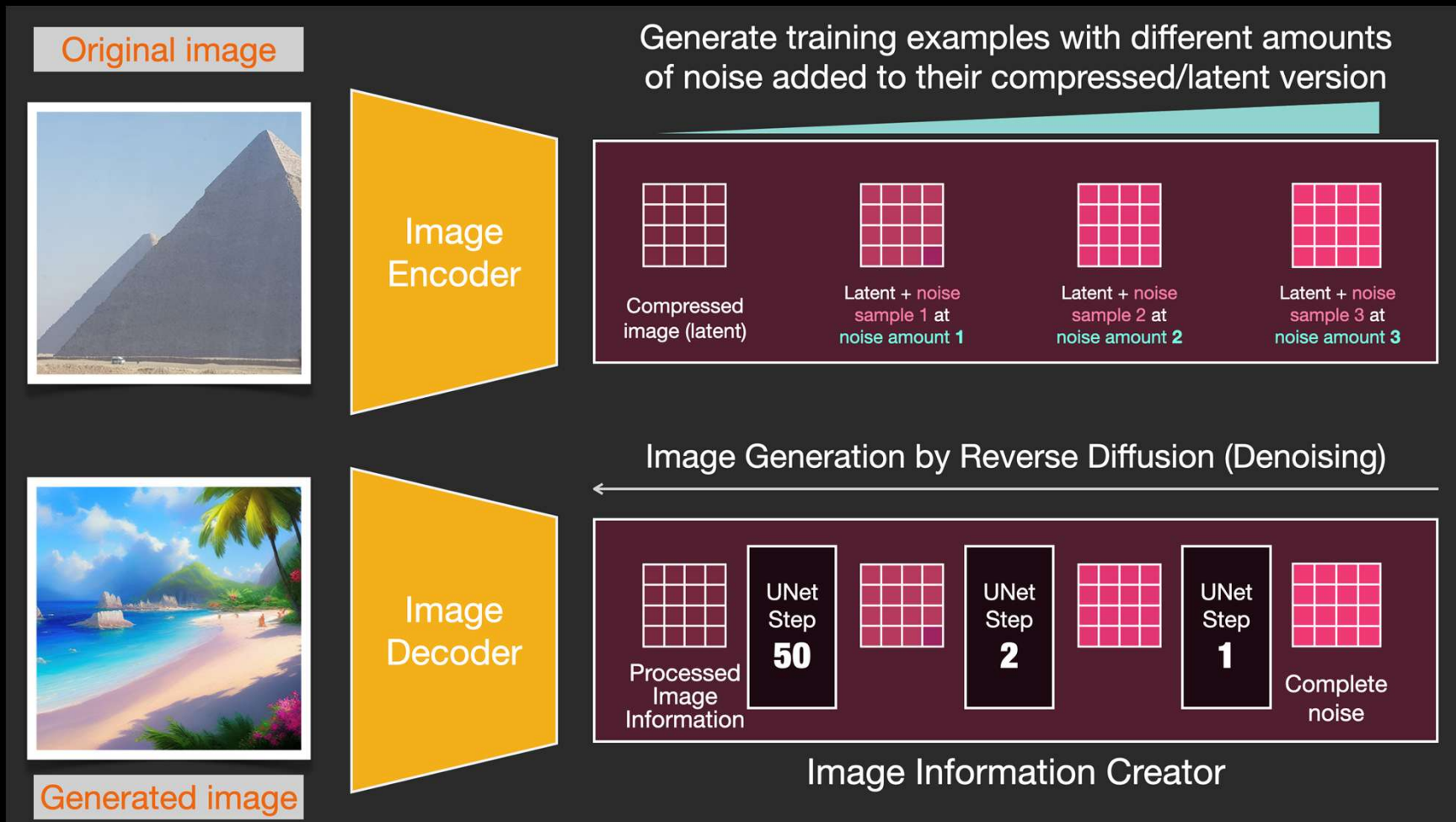
<https://x.com/AnjneyMidha/status/1989387242083950728?s=20>

Stable Diffusion – image generation by de-noising



<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion – image generation by de-noising



<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion – leveraging CLIP

Contrastive Language Image Pre-training (CLIP) is trained on a dataset of images and their captions (~400 million images and their captions)

IMAGE



Photo pour Japanese
pagoda and old house in
Kyoto at twilight - image
libre de droit

CAPTION



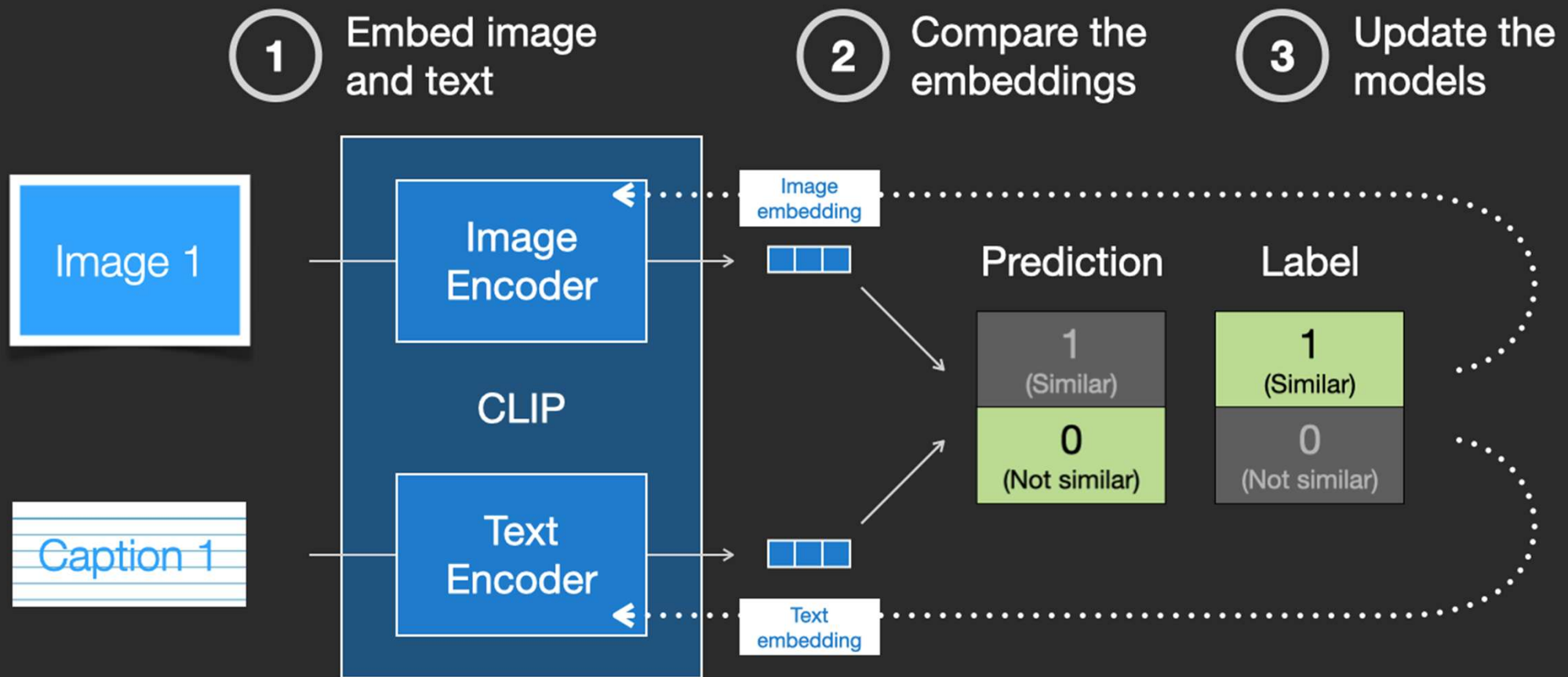
Soaring by Peter
Eades



far cry 4 concept art is the
reason why it 39 s a
beautiful game vg247.
Black Bedroom Furniture
Sets. Home Design Ideas

<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion – adding text embeddings encoder



<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion – adding text embeddings encoder



<https://jalammar.github.io/illustrated-stable-diffusion/>

Stable Diffusion demo and assignment

Speech to Text Models

Open AI **Whisper**

Uses a Transformer-based encoder-decoder architecture trained on a large dataset of diverse audio.

Google Speech to Text

Google's cloud service for speech recognition.

Based on large-scale neural network models (originally LSTM/RNN-based, now moving towards Transformer-based architectures).

Commercial services:

Amazon Transcribe, Microsoft Azure Speech, Apple Siri

<https://huggingface.co/spaces/openai/whisper>

Text to speech models – how do they work?

Text Analysis / Front-End

The input text is normalized (expanding abbreviations, handling numbers, punctuation),

Acoustic Model

A neural network (e.g., Tacotron, FastSpeech, or VITS) predicts an intermediate acoustic representation—commonly a mel-spectrogram (similar to embeddings creation)

Vocoder

A separate model (e.g., WaveNet, HiFi-GAN, WaveRNN) that converts the spectrogram to raw audio waveforms.

Post-Processing

The generated audio can be enhanced or normalized (e.g., for loudness), voice style transfer or emotional prosody control, which modifies rhythm and intonation for different contexts.

Text to speech models

- Tacotron (encoder–decoder approach) +vocoder
- Vocoder (e.g., WaveGlow, Deepmind Wavenet) transforms the “predicted” acoustic features into the actual waveform you hear.
Wavenet is one of initial vocoders, trained on audio data, can synthesize highly realistic speech.
- VITS (Variational Inference TTS)
- Commercial services
Amazon Polly, Microsoft Azure TTS, Google Cloud TTS

Multi modal models

Multimodal input refers to the ability of AI systems to accept and process and generate different types of data simultaneously, such as text, images, audio, and video.

Examples:

GPT-4, Gemini, DALL-E, Stable diffusion

Comparing the image generation architectures (1/3)

Feature	GAN (Generative Adversarial Network)	VAE (Variational Autoencoder)	Stable Diffusion
Architecture	Generator and Discriminator in adversarial setup	Encoder-Decoder with probabilistic latent space	U-Net with noise diffusion in a latent space
Working Principle	Generator creates images; Discriminator judges real vs fake	Encodes input into a latent distribution and reconstructs it	Iterative denoising from noise to image
Training Complexity	High; adversarial training can be unstable	Moderate; stable but can lead to blurry outputs	High; stable but computationally intensive
Inference Complexity	Low; single forward pass	Low; single forward pass	High; multiple denoising steps
Training Cost	High due to adversarial competition	Low to moderate	Very high due to iterative learning
Inference Cost	Low	Low	High; slower due to iterative refinement

Comparing the image generation architectures (2/3)

Feature	GAN (Generative Adversarial Network)	VAE (Variational Autoencoder)	Stable Diffusion
Output Quality	High, but prone to mode collapse	Moderate; often blurry	Very high; detailed, sharp, and diverse
Diversity of Outputs	Limited due to mode collapse	Moderate	High; better at generating unique and varied results
Latent Space	Unstructured and harder to navigate	Smooth and continuous	Compressed, structured, and interpretable
Robustness	Low; sensitive to hyperparameters	High	High; consistent across conditions
Main Use Cases	Image generation, face generation, deepfakes	Anomaly detection, data compression, reconstruction	Text-to-image, image editing (inpainting, outpainting), art generation
Conditional Generation	Challenging to implement	Limited	Built-in; excels in text-to-image generation

Comparing the image generation architectures (3/3)

Feature	GAN (Generative Adversarial Network)	VAE (Variational Autoencoder)	Stable Diffusion
Mode Collapse Risk	High	Low	Very low
Training Time	Long	Short to moderate	Very long
Interpretability	Low	High	High
Resolution Support	High, but less effective for very large images	Low to moderate	High; built for high-resolution generation
Realism of Output	High, photorealistic	Moderate; less sharp	Very high; fine details and textures
Noise Tolerance	Low	Moderate	High; denoising is part of the process
Deployment Ease	Easy after training	Easy	Complex due to high inference cost
Energy Efficiency	Moderate	High	Low; requires significant GPU resources



SORA DEMO:

<https://openai.com/sora>

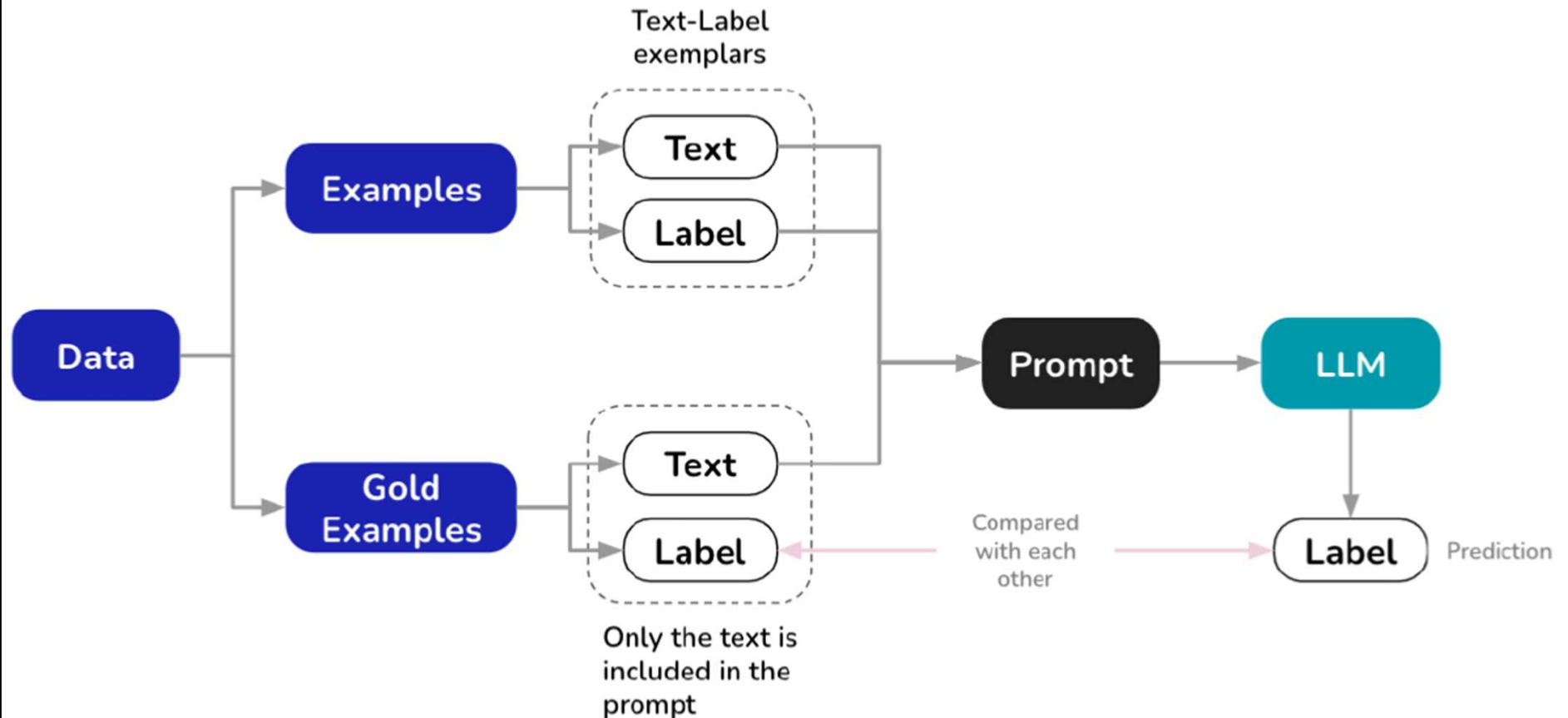
Figure 01 Robot demo

<https://www.youtube.com/watch?v=Sq1QZB5baNw>

Generative Models Quiz

LLMs for classification and generation tasks

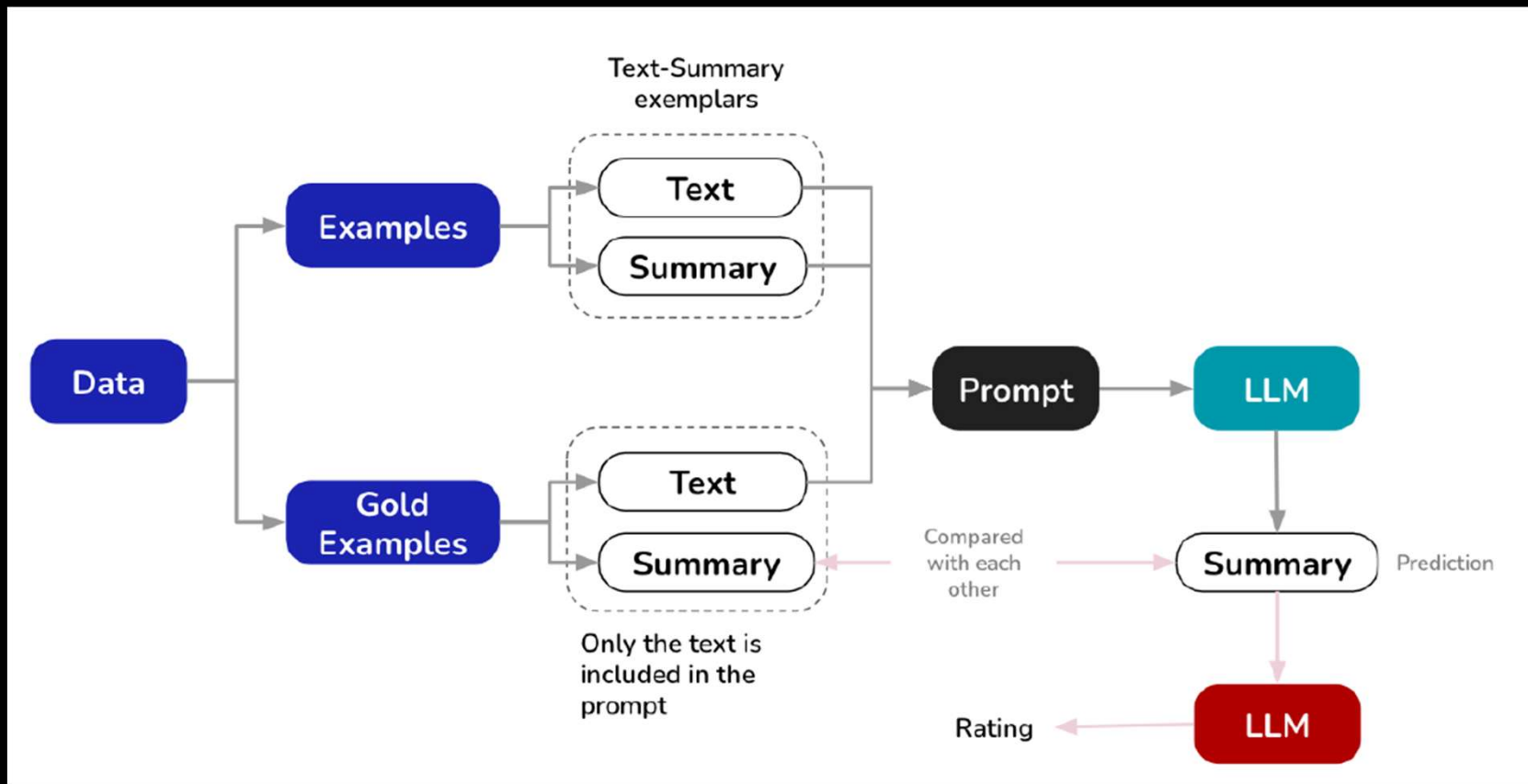
LLMs for classification tasks - overall flow



Classification Hands-on

- Sentiment Analysis
- Aspect-based Sentiment Analysis

LLMs for generation tasks - approach



Reasoning in LLMs

- Based on pattern matching

How does it seem to be “reasoning”?

Consider following chat:

whats the weather in chennai?

The current weather in Chennai is clear sky with a temperature of 23.74°C.

what should i see there?

<thinking> The User is asking for suggestions on what to see in Chennai. However, this type of request is not directly related to the current weather and cannot be answered using the provided "get_weather" tool. I need to provide general information based on what I know. </thinking>

Chennai, also known as Madras, is a vibrant city with many attractions.

If LLM is doing next token prediction, how is it thinking/reasoning?

Do LLMs actually reason?

There is no internal conscious reasoning happening.

What looks like “thinking” is actually generated text that imitates **reasoning patterns learned during training**.

During training, it saw millions of examples formatted like:

```
<thinking>
step-by-step reasoning
</thinking>
final answer
```

```
<thinking>
reason about the task
</thinking>

<tool_call>
call weather api
</tool_call>
```

The model learned a pattern:

1. Analyze request
2. Decide if tool is needed
3. Produce answer

So, it is **simulating reasoning**, not actually performing internal reasoning.

So does that mean model can never invent anything new?

Not really, the model can combine multiple patterns it has learnt to produce a new pattern

Example:

Training data might contain:

- electric cars
- Drones
- delivery services

The model could invent:

A drone network that swaps EV batteries for delivery trucks mid-route.

That exact idea may never exist in the training data, but it is created by combining learned patterns.

But there is a limit to which it can “invent” something new

LLMs cannot easily create true paradigm-breaking ideas because:

- They depend on training distribution.
- They lack real-world experimentation.
- They lack goals or curiosity.

So they are excellent at **recombination creativity**, but weaker at **discovery creativity**.

Note: While Agents can enable the LLM to do real world experimentation, but note that the output of that experiment is not fed back into the LLM for learning. (as weights are frozen in a trained model)

Deductive reasoning – top-down

Deductive reasoning applies a general, established rule to a specific situation to reach a guaranteed conclusion.

Core Idea

General Rule → Specific Case → Guaranteed Conclusion

Rule: You must wash hands before eating food

Specific case: I am going to eat food

Conclusion: You must wash hands

LLM Context

LLMs excel at deductive reasoning because it is fundamentally about applying learned patterns and rules from their vast training data. They can act like a logical processor if given a clear set of premises

Deductive reasoning – example

Prompt:

Here is a rule: All patients who undergo cardiac bypass surgery must be monitored for at least 24 hours.
Patient John Smith underwent cardiac bypass surgery this morning, on June 9, 2025.
Based on this rule, what is the minimum monitoring requirement for John Smith?

Expected LLM Output:

Based on the rule provided, the minimum monitoring requirement for John Smith is to be monitored for at least 24 hours.

- **Why it's deductive:** The LLM applies the general rule (all bypass patients) to the specific case (John Smith) to reach a logically certain conclusion.

Inductive reasoning – bottom-up

Inductive reasoning observes specific examples or patterns to form a general, likely conclusion. It's about making an educated guess based on trends.

Core Idea

Specific Observations → Pattern Identification → Likely General Conclusion

Observations: Most people who ate dinner washed their hands before eating

Scenario: I just ate my dinner

Conclusion by LLM: You must have washed your hands

LLM Context

This is a core strength of LLMs. Their training process is itself a massive act of induction, where they analyze billions of specific text examples to form general patterns about language, facts, and concepts.

They use this ability to summarize, categorize, and predict.

Inductive reasoning – example

Analyze the following snippets from recent customer reviews for a new restaurant

- * Review 1 (Monday): "Incredible food! The chef is a genius. Service was attentive"
- * Review 2 (Tuesday): "Five stars! Best pasta I've ever had. It was a calm and ro"
- * Review 3 (Saturday): "The food was good, but it took 45 minutes to get our drin"
- * Review 4 (Saturday): "Waited 30 minutes for our table despite having a reservat
- * Review 5 (Wednesday): "A delightful and peaceful mid-week dinner. Flawless serv

What is the likely experience for a new customer on a Saturday night?

Based on the pattern in the reviews, a new customer visiting 'The Crimson Fork'

- * ****Food Quality:**** The food will likely be excellent.
- * ****Atmosphere:**** The restaurant will probably be very busy, crowded, and loud.
- * ****Service:**** They will likely experience slow service and potential waits, as

In summary, while the food is consistently praised, the overall experience on a

Why it's inductive: The LLM identifies a pattern from specific reviews—great food always, but chaotic service specifically on weekends—and generalizes to predict a future customer's likely experience.

Abductive reasoning – inference to the best explanation

Abductive reasoning involves making the most plausible guess to explain an observation. It's the reasoning we use for troubleshooting and diagnostics.

Core Idea

Incomplete Observation → Best Possible Explanation (Hypothesis)

Example

Observation: “puddle under the car”

Conclusion: It might have rained, or AC condensation

LLM Context

LLMs use abductive reasoning to generate plausible hypotheses or fill in missing information.

When faced with an ambiguous query, the model pieces together **context** to provide the most relevant or likely answer.

Abductive reasoning – example

An LLM is given the observation:
"The lawn is wet."

It then generates possible explanations:

- "It rained."
- "The sprinklers were on."
- "Someone watered the lawn manually."

Through abductive reasoning, the LLM infers the most likely explanation **based on its training data and context**.

If it has access to weather data and knows it rained recently, it might conclude "It rained" as the most probable cause.

If it knows the user's typical watering schedule, "The sprinklers were on" might be prioritized.

Analogical reasoning – reasoning by comparing

Analogical reasoning explains a new or complex idea by comparing it to something familiar.

Core Idea

Thing A is like Thing B, so let's use our understanding of B to understand A.

Prompt: Explain dollar cost averaging in simple terms

Response: Think of dollar cost averaging like regularly buying groceries...

LLM Context

LLMs use analogy to explain complex topics in simpler terms. They can map the structure of one domain onto another, making abstract concepts more concrete and relatable for the user.

Analogical reasoning – example

Prompt: “A bird uses wings to fly. Give me a similar example”

Response: Fish uses fins to swim

Trivia: If it is asked

Prompt: “Would following analogy also be valid based on above: Planes use wings to fly”

A “smart” LLM will respond like so:

To some extent. A bird uses its **wings to generate lift and propulsion** for flight. A plane uses its **engines to generate thrust** for propulsion, and its **wings are designed to generate lift** for flight.

Summarization: Template based summarization

Summarize a document or note and produce a template based structured output.

For example, a doctor's note containing both unstructured and structured text:

Patient: John Doe, MRN: 789123, Date of Visit: 2025-06-09, Physician: Dr. Emily Carter

Notes: John, a 58-year-old male, presents for a follow-up on his type 2 diabetes and hypertension. He reports good adherence to his medications (Metformin 1000mg twice daily and Lisinopril 20mg daily). He has been monitoring his blood sugar at home, with fasting levels ranging from 110-130 mg/dL. He denies any chest pain, shortness of breath, or headaches. He mentions occasional dizziness when standing up too quickly. He is trying to follow a low-sodium, low-sugar diet but finds it challenging during weekends.

Summarization: Abstractive summarization

Create an concise summary of a document, capturing core points

For example:

Summarize the budget speech

Abstractive summarization with streaming data

[2025-06-09 08:00:00] Patient admitted to ICU. HR: 110 bpm, BP: 90/60 mmHg, SpO2: 92% on room air, Temp: 101.5°F. Patient is lethargic.

[2025-06-09 08:30:00] Oxygen started at 2L via nasal cannula. HR: 105 bpm, BP: 95/65 mmHg, SpO2: 95%

[2025-06-09 09:00:00] IV fluids initiated. HR: 100 bpm, BP: 100/70 mmHg, SpO2: 96%. Patient is more alert.

[2025-06-09 09:30:00] HR: 98 bpm, BP: 105/72 mmHg, SpO2: 97%.

[2025-06-09 10:00:00] Temp: 100.8°F. HR: 95 bpm, BP: 110/75 mmHg, SpO2: 98%.

Extractive summarization

Structured info in json or xml from patient notes

Abstractive summarization with streaming data

[2025-06-09 08:00:00] Patient admitted to ICU. HR: 110 bpm, BP: 90/60 mmHg, SpO2: 92% on room air, Temp: 101.5°F. Patient is lethargic.

[2025-06-09 08:30:00] Oxygen started at 2L via nasal cannula. HR: 105 bpm, BP: 95/65 mmHg, SpO2: 95%

[2025-06-09 09:00:00] IV fluids initiated. HR: 100 bpm, BP: 100/70 mmHg, SpO2: 96%. Patient is more alert.

[2025-06-09 09:30:00] HR: 98 bpm, BP: 105/72 mmHg, SpO2: 97%.

[2025-06-09 10:00:00] Temp: 100.8°F. HR: 95 bpm, BP: 110/75 mmHg, SpO2: 98%.

Abstractive summarization with streaming data

[2025-06-09 08:00:00] Patient admitted to ICU. HR: 110 bpm, BP: 90/60 mmHg, SpO2: 92% on room air, Temp: 101.5°F. Patient is lethargic.

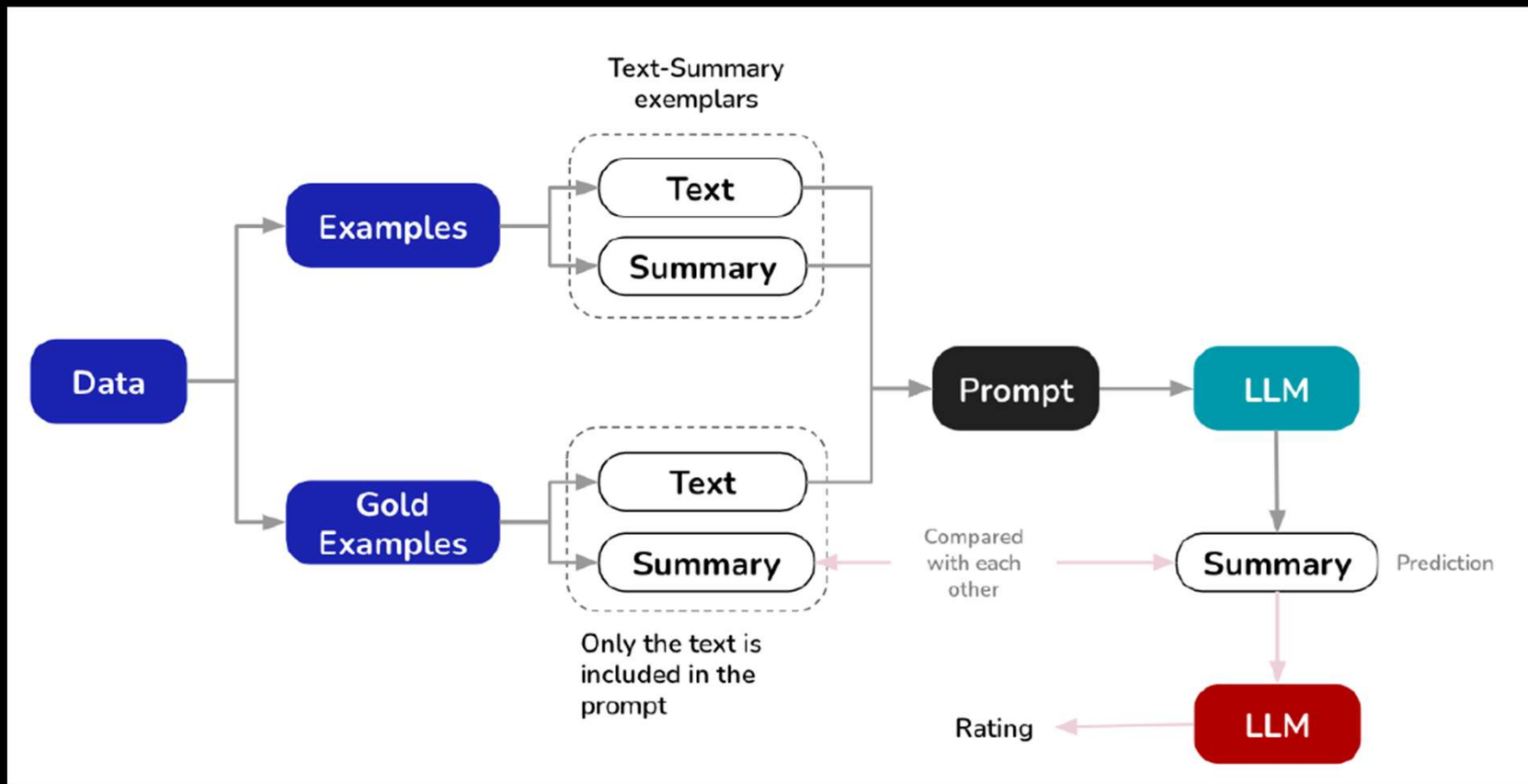
[2025-06-09 08:30:00] Oxygen started at 2L via nasal cannula. HR: 105 bpm, BP: 95/65 mmHg, SpO2: 95%

[2025-06-09 09:00:00] IV fluids initiated. HR: 100 bpm, BP: 100/70 mmHg, SpO2: 96%. Patient is more alert.

[2025-06-09 09:30:00] HR: 98 bpm, BP: 105/72 mmHg, SpO2: 97%.

[2025-06-09 10:00:00] Temp: 100.8°F. HR: 95 bpm, BP: 110/75 mmHg, SpO2: 98%.

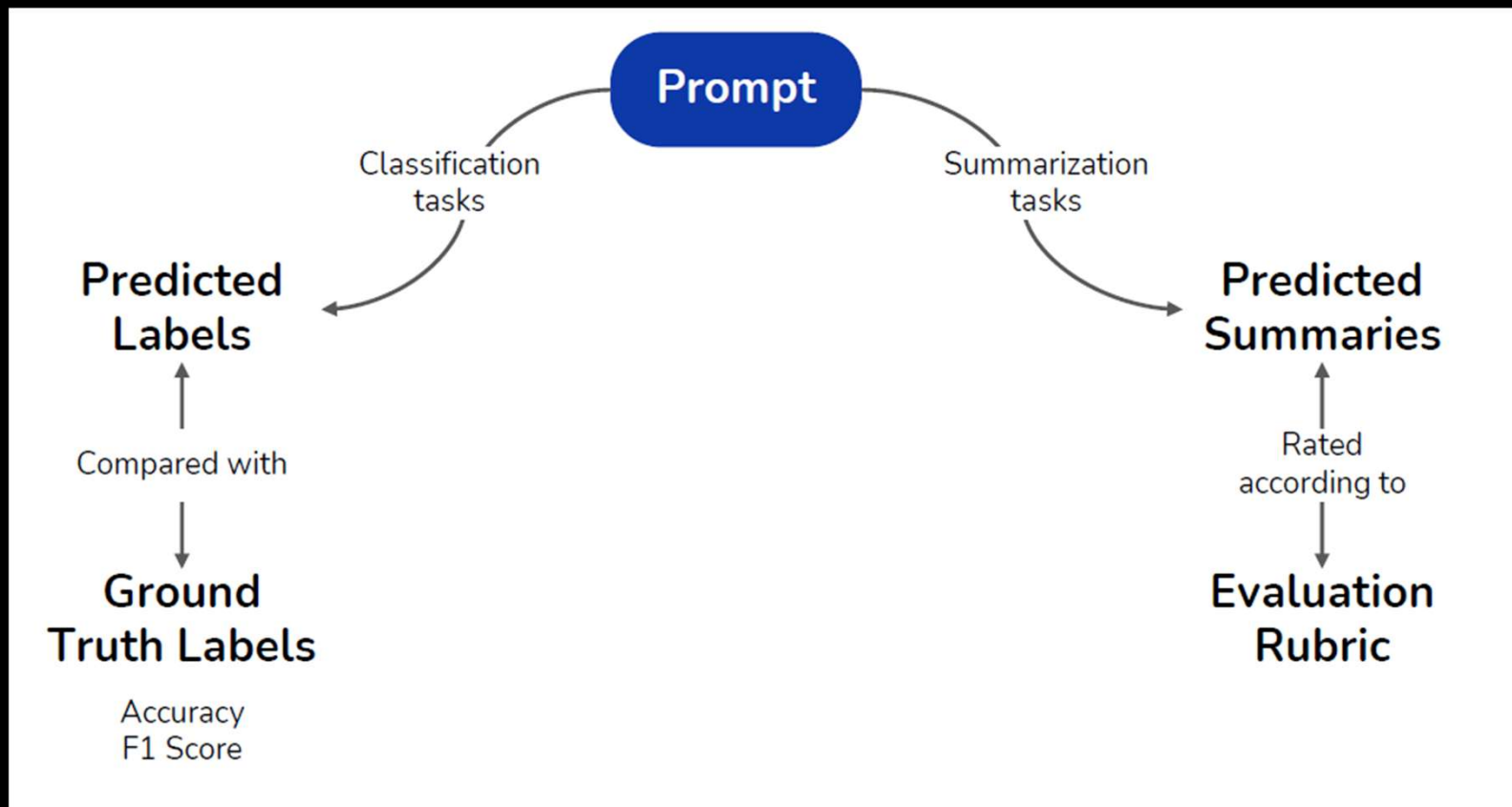
LLMs for generation tasks - approach



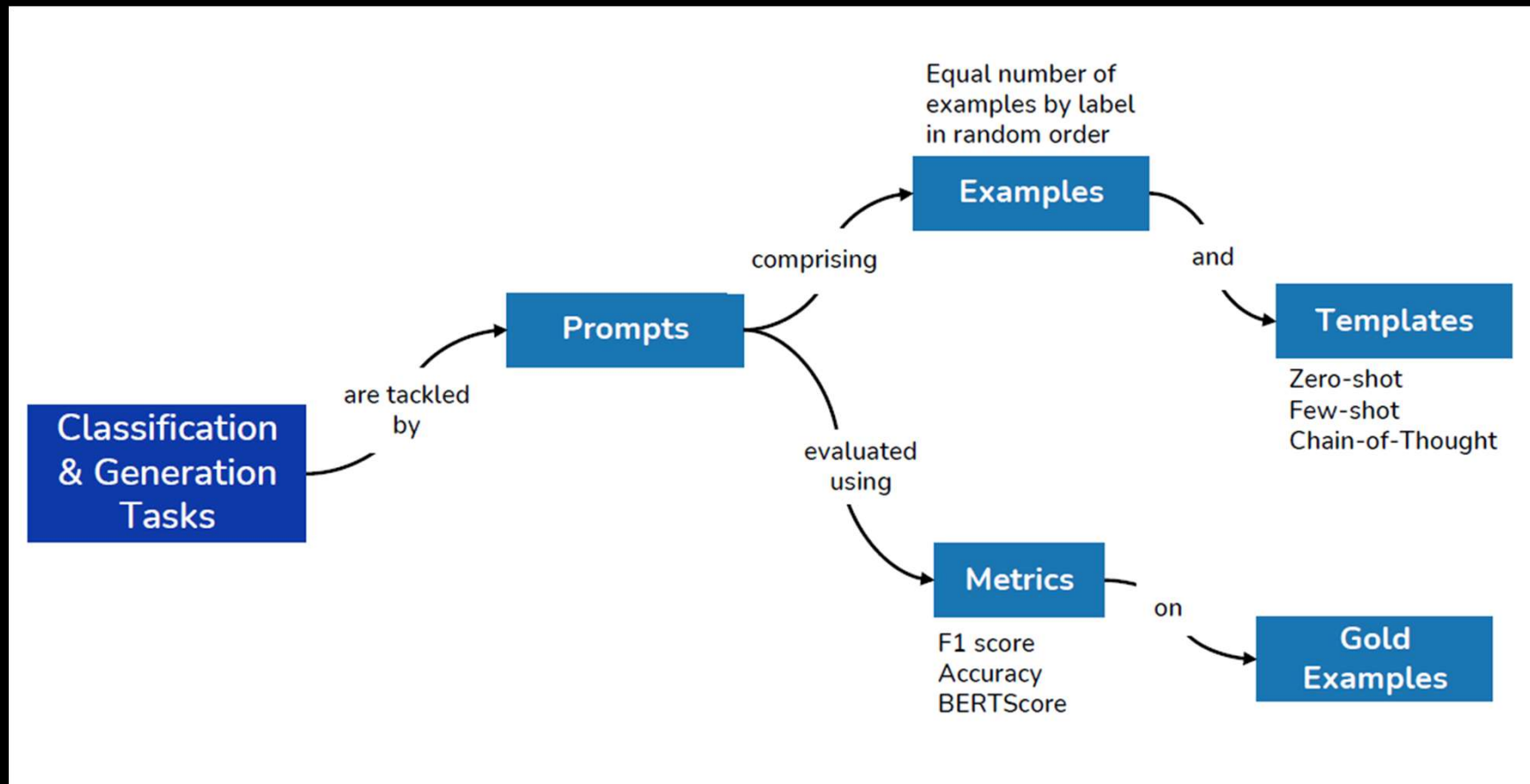
Generation Hands-on

- Dialogue Summarization

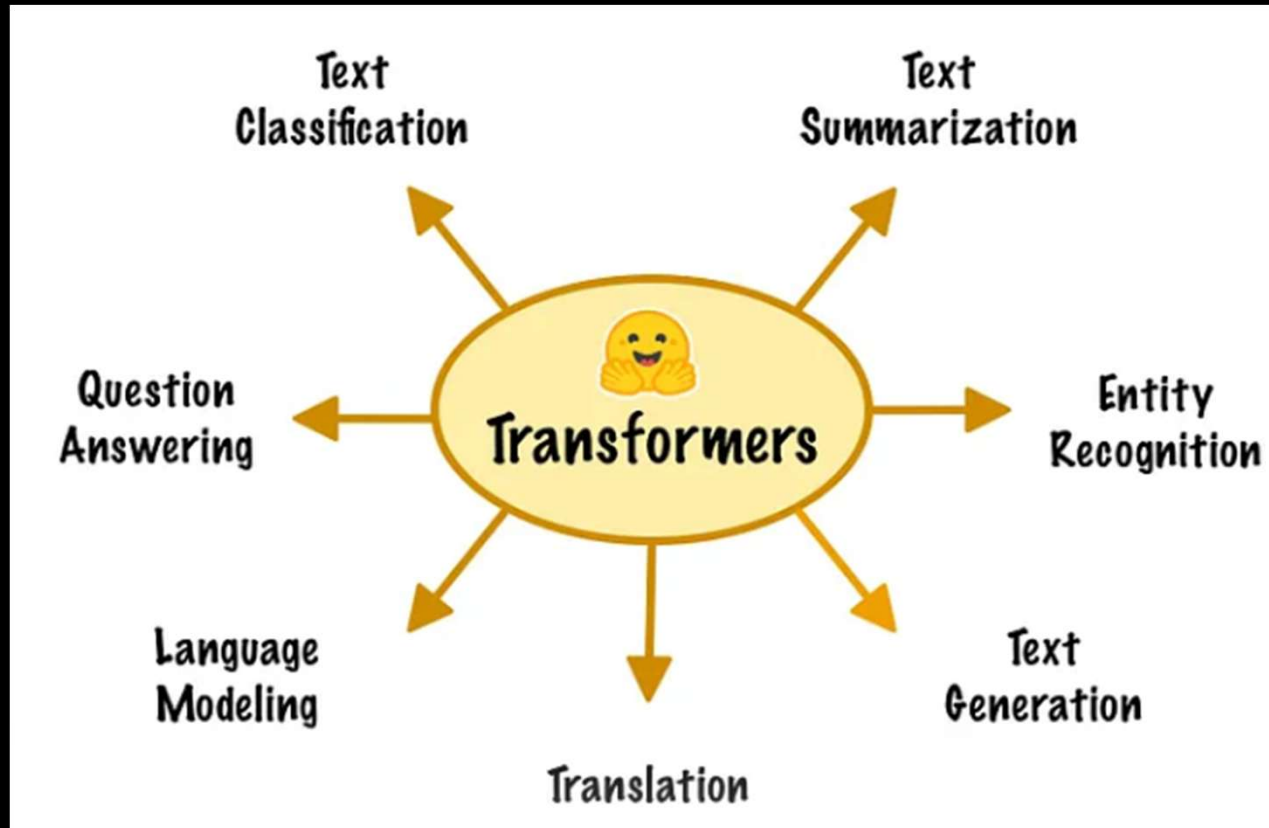
Evaluation for classification and generation



Summary of full flow



Core Capability: NLP



Core capability - inferencing

Inferencing involves generating outputs based on **patterns** and **knowledge** learned during their **training**. This can include answering questions, completing sentences, translating text, and more

Prompt: "What is the capital of France?"

Response: "Paris."

Prompt: A detailed article about climate change.

Response: "Climate change is a significant global issue caused by human activities leading to rising temperatures and environmental impacts."

Translation: Translating text from one language to another.

Prompt: "Hello, how are you?" (English)

Response: "Hola, ¿cómo estás?" (Spanish)

Text Generation: Producing creative content such as stories, poems, or essays.

Prompt: "Once upon a time in a faraway land,"

Response: "there was a small village surrounded by towering mountains and lush forests."

Sentiment Analysis: Determining the sentiment or emotional tone of a text.

Example: Input: "I love this product!"

Output: "Positive sentiment."

Core capability - summarization

It can summarize large documents, videos, images, flowcharts, audio – any modality, multiple languages

Extractive Summarization:

The LLM selects and concatenates key sentences or phrases from the original text

Abstractive Summarization:

The LLM generates new sentences that paraphrase the original content.

Practical applications

News summarization, scientific papers summarization, legal documents, product reviews

Challenges and limitations:

May miss critical details or emphasize less important information affecting comprehensiveness

Coherence and relevance: While generally good, can be lacking sometimes

Bias and errors: Biases present in training data can result in skewed or inaccurate summaries

Core capability - reasoning

Reasoning in AI involves higher-level cognitive processes (compared to inferencing) where the AI system makes decisions based on logic, understanding relationships, and drawing conclusions from given information.

Deductive reasoning:

All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.

Inductive reasoning

Every time I water my plant, it grows taller. Watering plants regularly helps them grow taller.

Abductive reasoning

Ability to choose best possible inference. Lawn is wet – rain? Or sprinkler?

Practical applications

Analyse market data to analyse patterns, customer behaviour analysis, drug discovery

Core capability – generate **new, unseen** outputs

Generate **new, unseen** text, image, audio, video

